Author's Hand

Autonomous AI Research Agent that summarises research papers — every claim traceable, linked to your ORCID, ResearchGate & Google Scholar

IIT Guwahati E&ICT Academy · Generative AI · Final Capstone Option 5 · LangChain · RAG · Agent · Streamlit + FastAPI · Windows app

Submitted by: Dr. Kameswari Jada

Too many papers, and AI summaries can’t be trusted

THE PROBLEM

Volume

Millions of papers a year; reading enough to stay current or write a review takes hours per paper.

Trust

Generic AI summarisers hallucinate numbers and give no way to check a claim against the source.

Context-blind

Tools ignore who you are — your ORCID record, themes, co-authors and gaps are never used.

THE SOLUTION

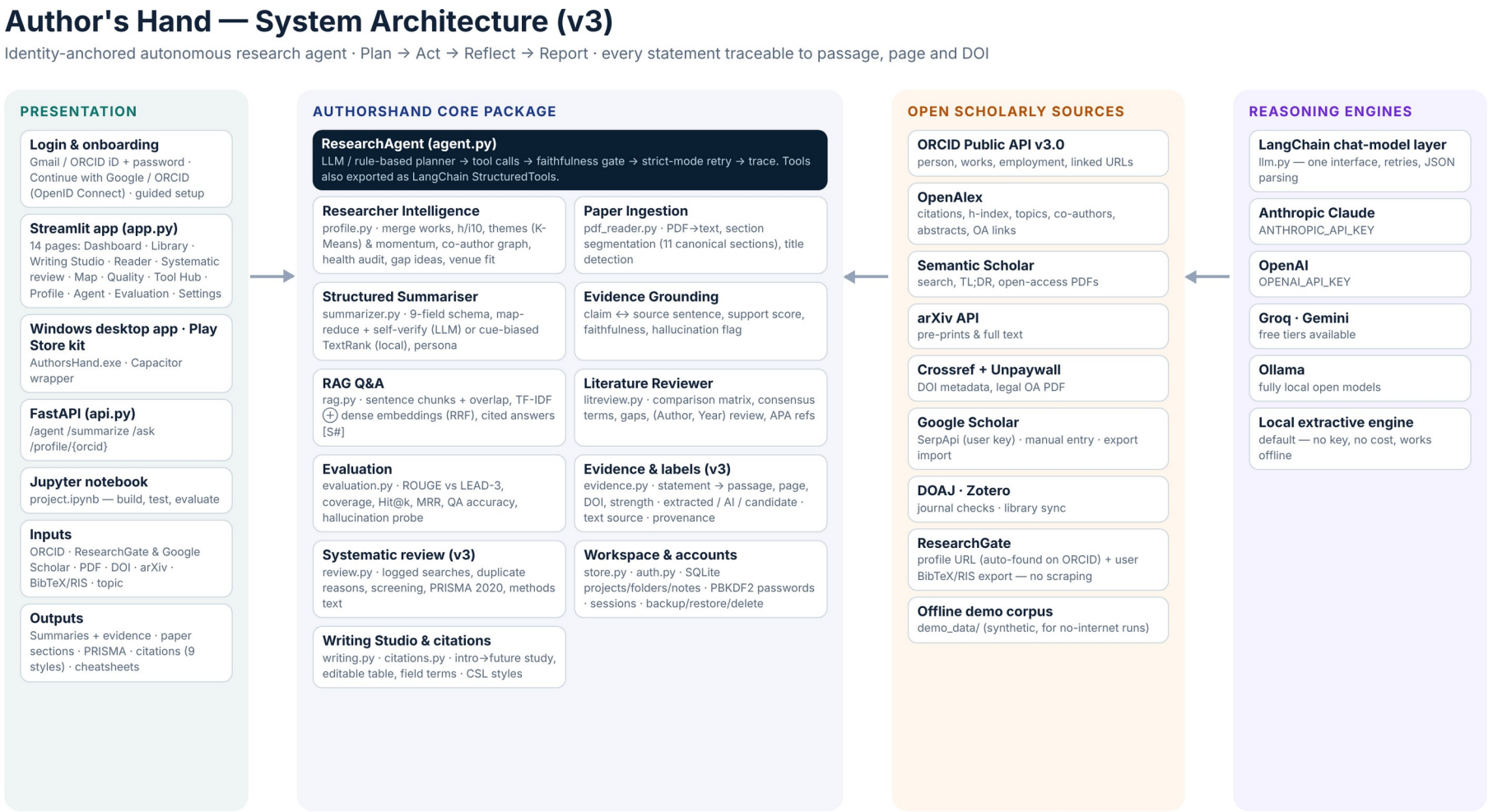
An agent that knows your research

- Plans from a plain-English goal and runs the full workflow
- 9-field structured summary with an evidence trail for every claim
- Self-reflects: re-summarises when evidence support is weak
- Links ORCID + ResearchGate → unified profile, themes, gaps, health audit
- Chat with papers (RAG) and auto literature review
- Free & offline by default; any LLM when available

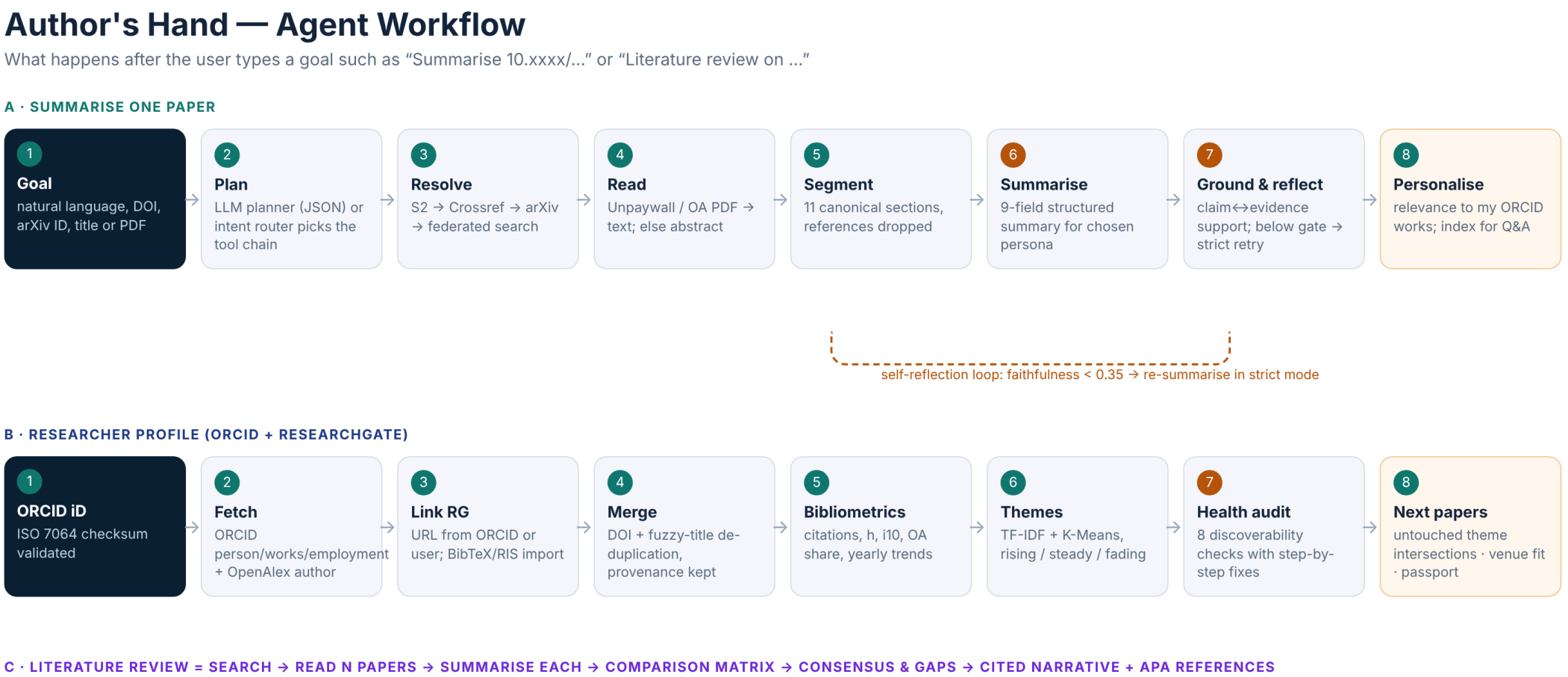
What Author's Hand does that market tools don’t

Capability	Typical summarisers	Author's Hand
Structured summary (problem → future work)	Partial	**9 fields · 3 personas**
Sentence-level evidence + faithfulness score	Rare	**Every claim linked to its source**
Self-reflection on weak summaries	No	**Quality gate + strict retry**
Linked to your ORCID & ResearchGate	No	**Unified, de-duplicated profile**
“How does this paper relate to my work?”	No	**Relevance score + your closest papers**
Theme momentum, gaps, next-paper ideas	No	**From your own record**
Profile-health audit with fixes	No	**8 checks, step-by-step**
Runs free / offline / any LLM	Mostly paid	**Local · Groq · Gemini · Ollama · Claude · OpenAI**

Architecture: four layers around one autonomous agent



Workflow: from a goal to a verified, personalised answer



How the agent thinks: Plan → Act → Reflect → Report

1

Plan

LLM planner returns a JSON intent + arguments; with no key a deterministic router does. Five intents: profile, summarise, review, venue, question.

2

Act

Calls tools — search, resolve DOI/arXiv, read open-access PDF, segment, summarise, index — logging thought → tool → result → time.

3

Reflect

Grounds every claim in a source sentence. Faithfulness below 0.35 triggers a strict-mode re-summary.

4

Report

Structured output, evidence trail and full trace; exports Markdown, BibTeX, CSV and a Research Passport.

Tech stack

Python · LangChain (chat models + StructuredTools) · scikit-learn (TF-IDF, K-Means) · NetworkX (TextRank, co-author graph) · pypdf · rouge-score · optional sentence-transformers · Streamlit · FastAPI · pytest

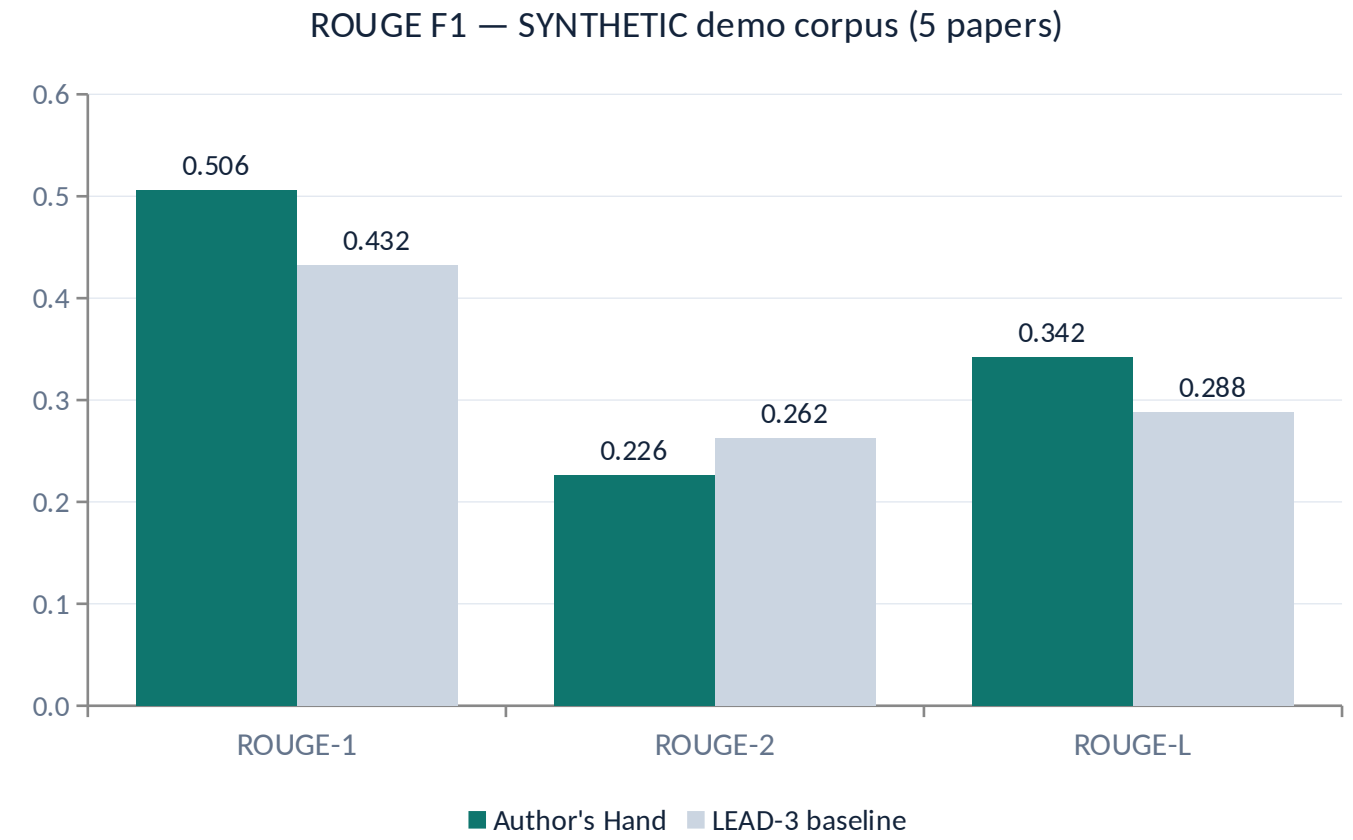
Results on the synthetic demo corpus (pipeline check, not real-world quality)

0.506
ROUGE-1
LEAD-3: 0.432

100%
Retrieval Hit@3
10 QA pairs · MRR 0.87

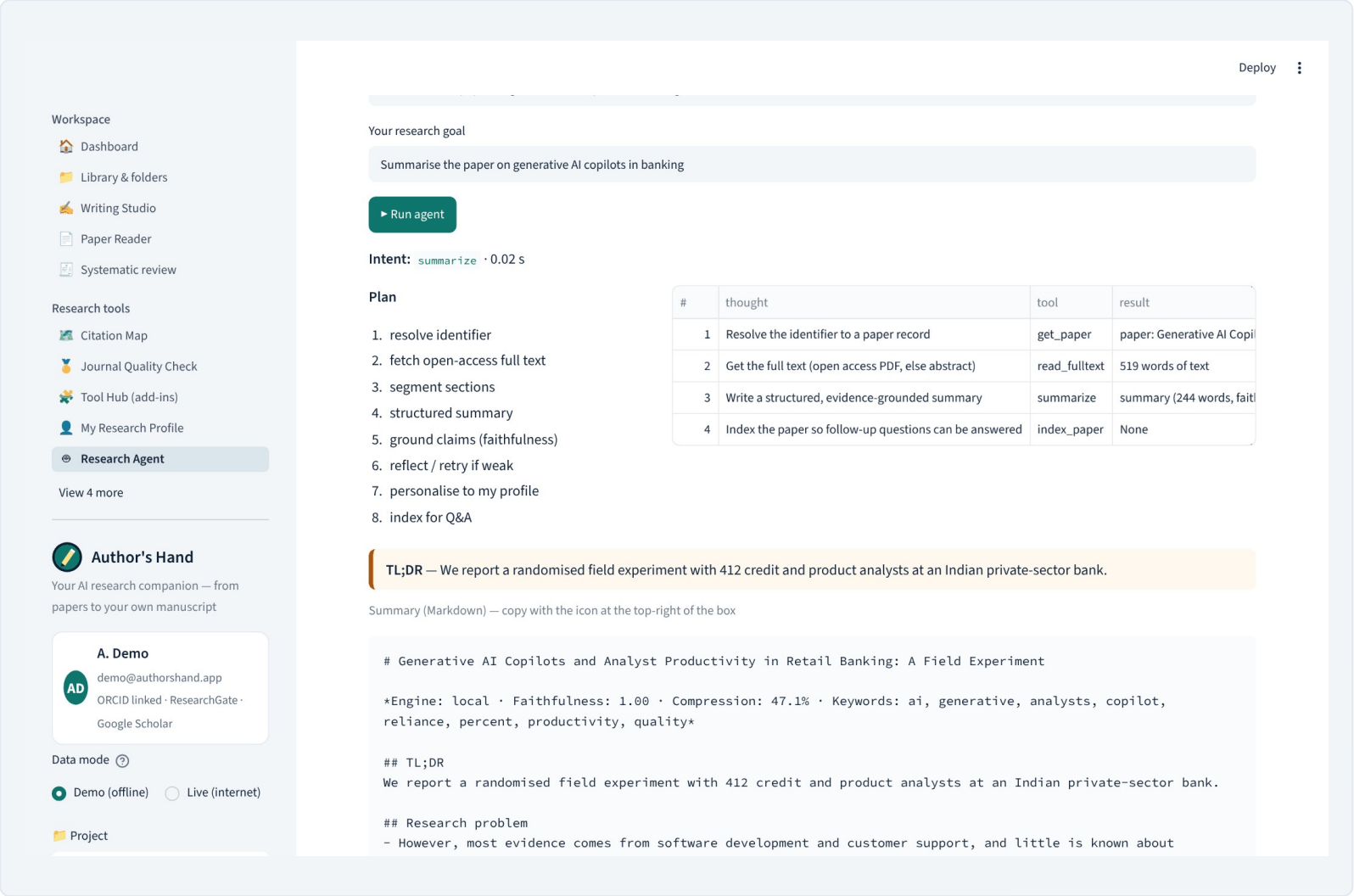
100%
Field coverage
9 structured fields

0.03
Support for an invented claim
vs 1.00 genuine → flagged



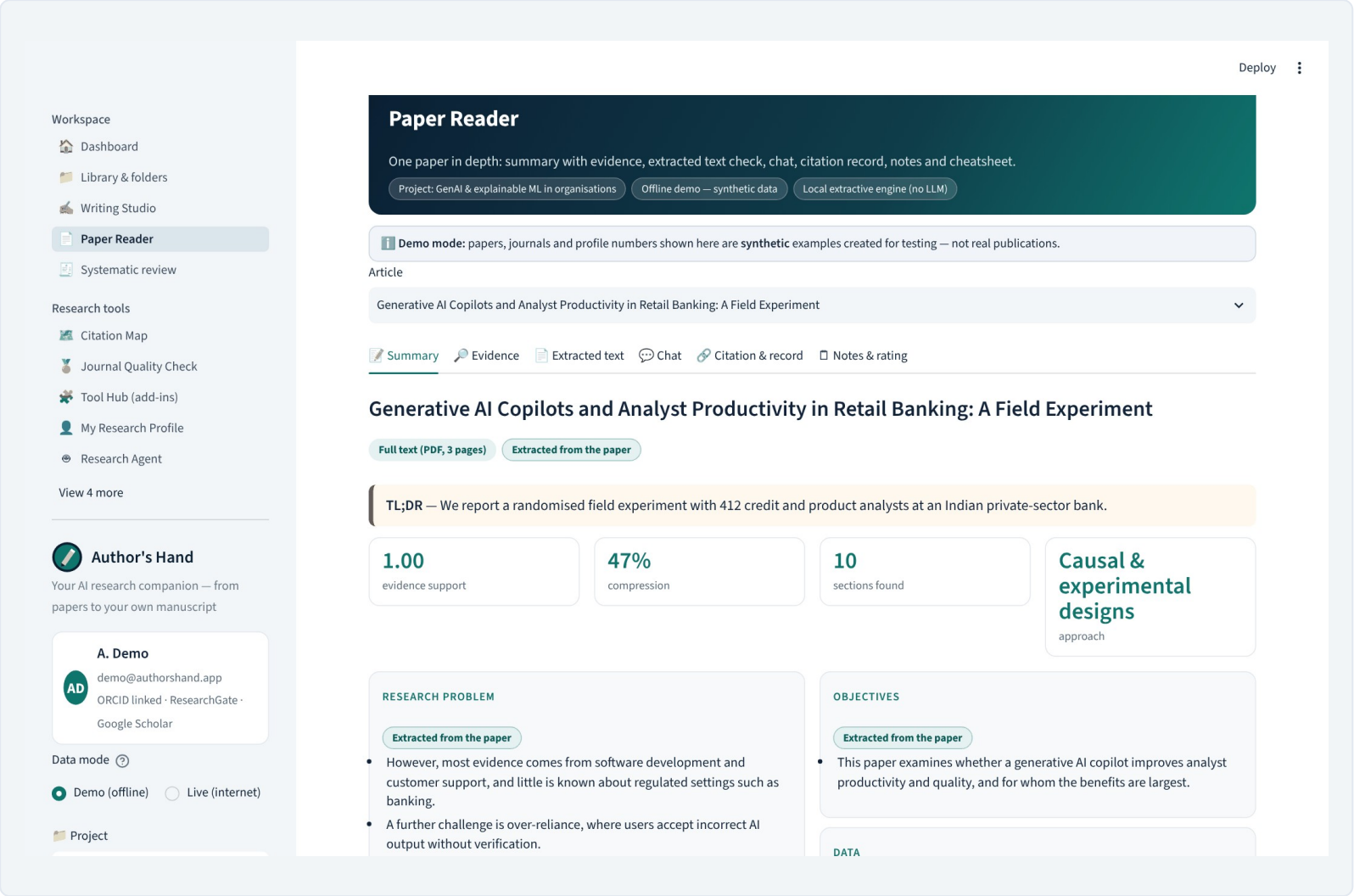
⚠️ Limitation: 5 synthetic papers written for testing — these numbers show the pipeline works, not quality on real literature. ROUGE-2 trails LEAD-3 (summary-like abstracts); local faithfulness is 1.0 by construction. Real-paper protocol + human ratings are built in (next slides). 33/33 unit tests pass.

Research Agent: a plan and an auditable trace



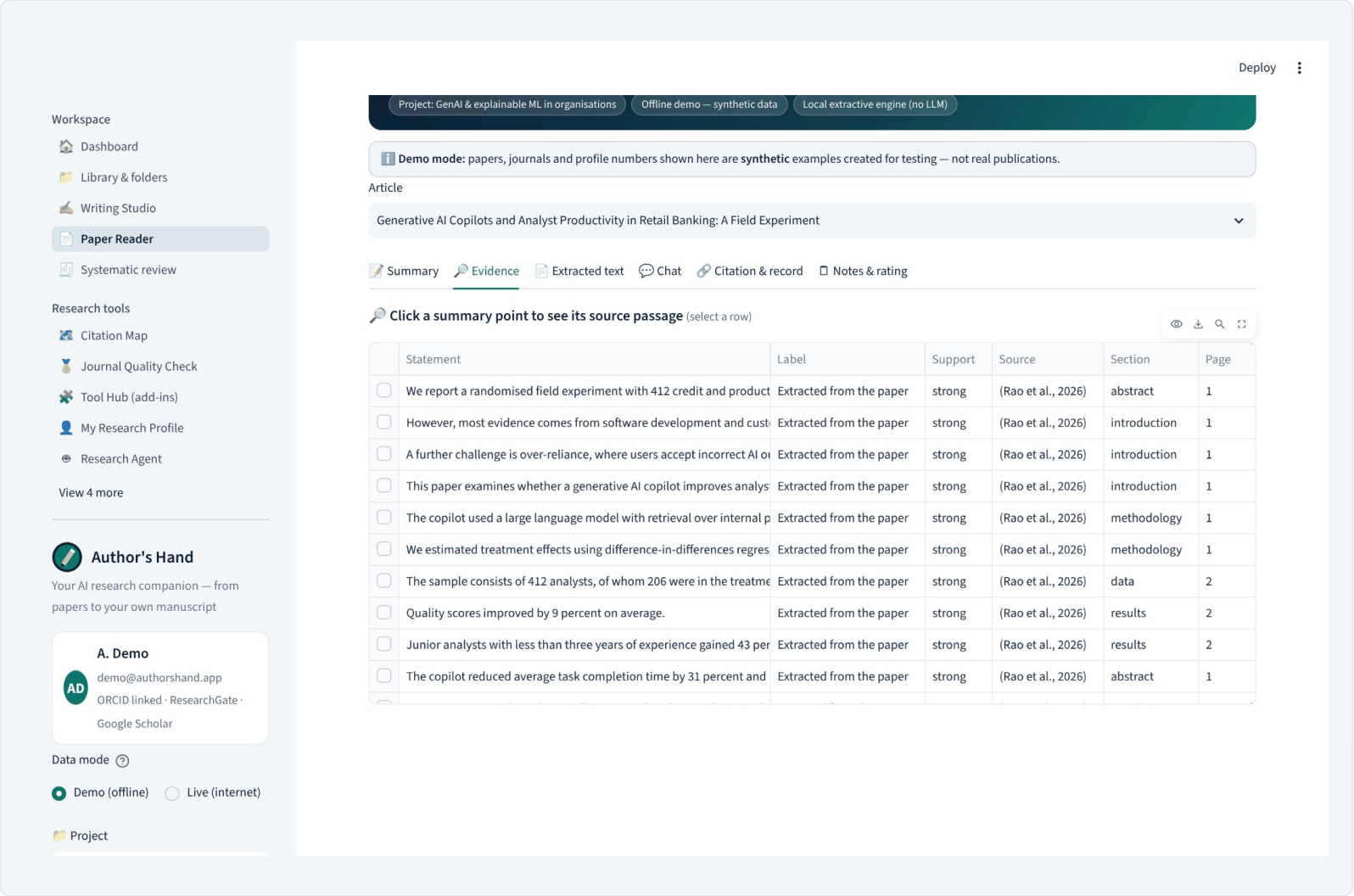
- Goal typed in plain English
- Intent detected, plan shown before results
- Each tool call logged with its reasoning, result and time
- Output: structured summary with evidence

Paper Reader: structured summary with labels



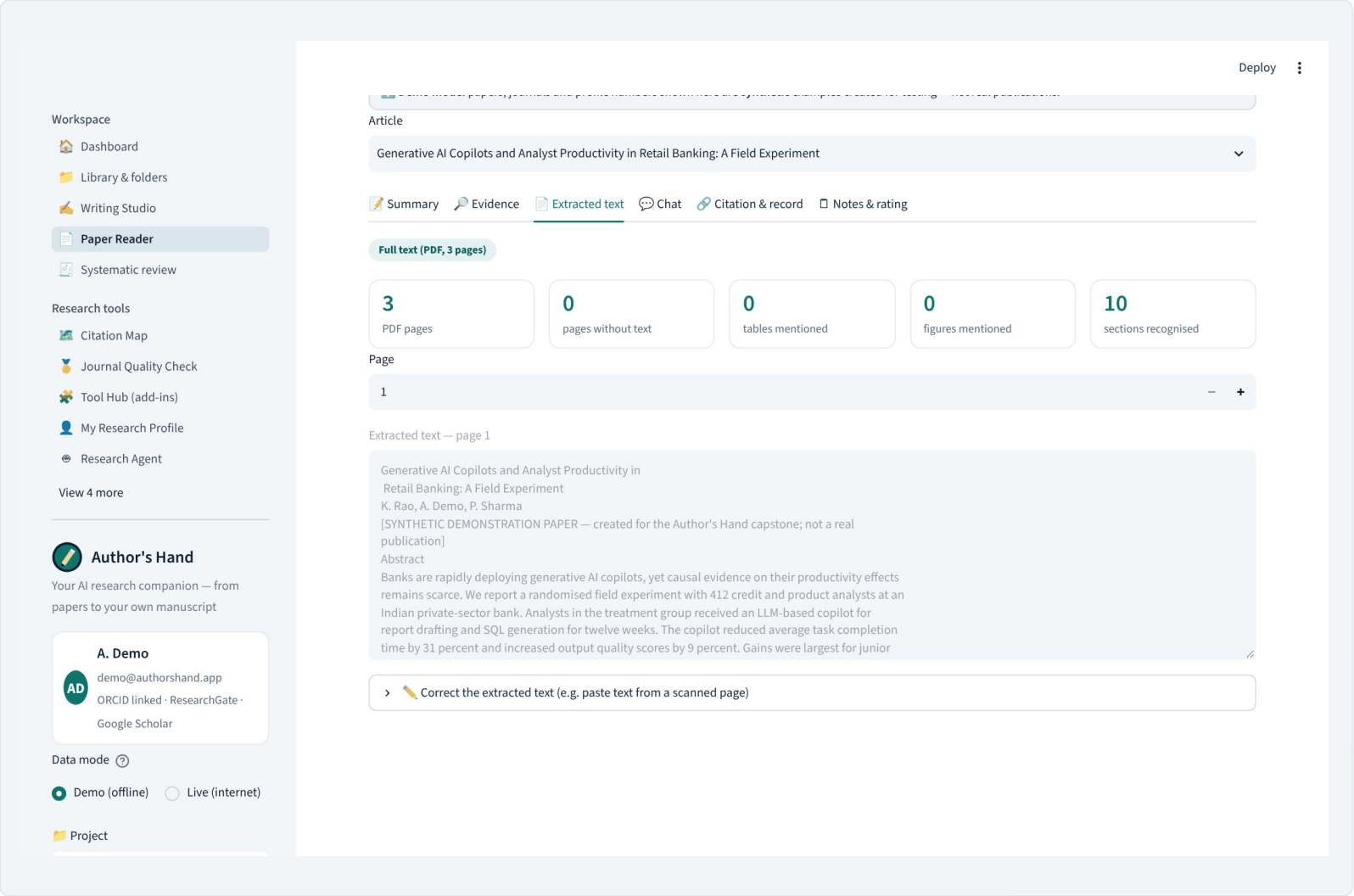
- Text-source chip: full text (PDF, 3 pages) vs abstract only
- Every point labelled “Extracted from the paper”
- Evidence support, compression, sections found
- Tabs: evidence, extracted text, chat, citation record, notes & rating

Evidence: click a statement → passage, section, page, DOI



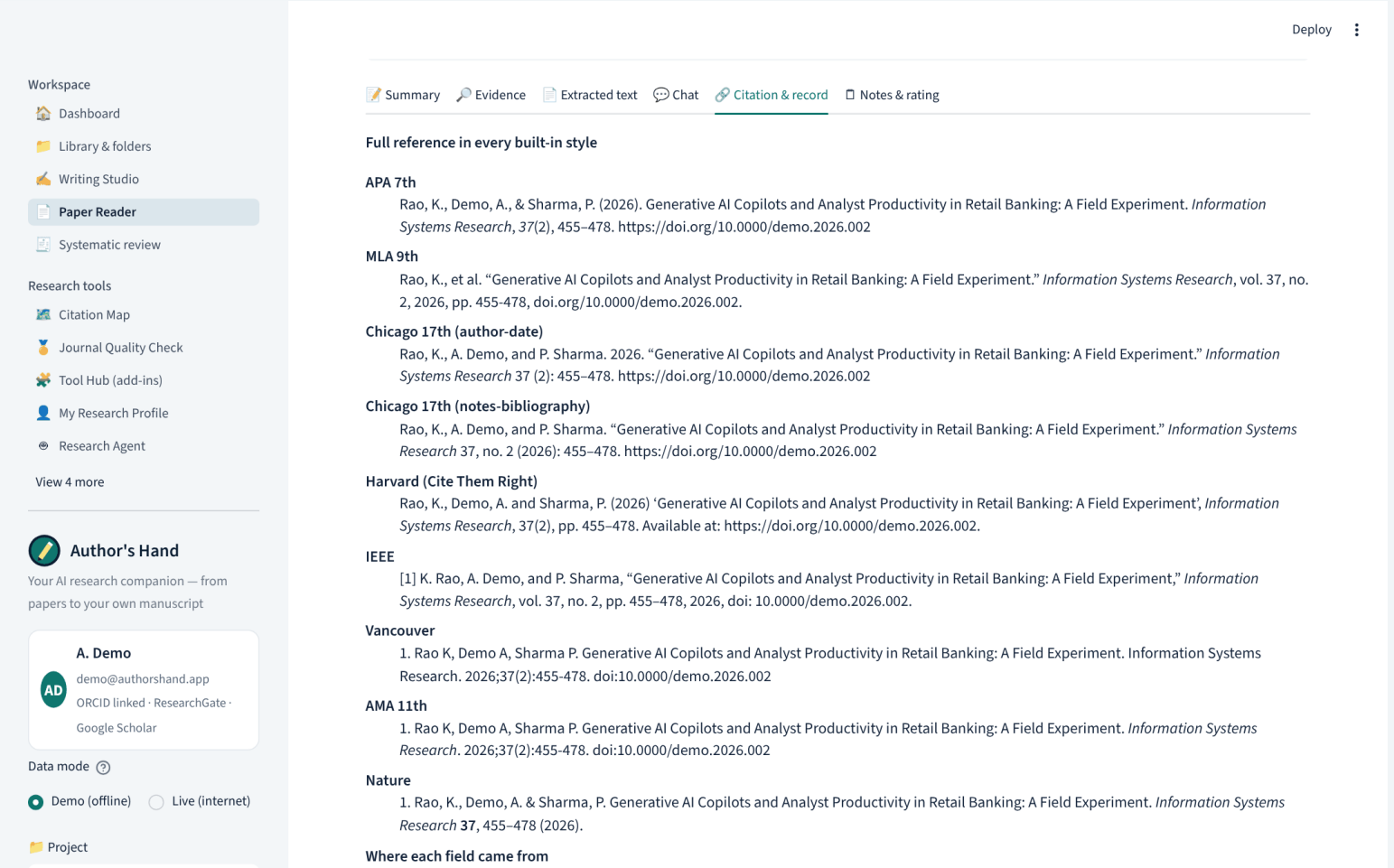
- Support strength: strong / moderate / weak
- Source passage with surrounding context
- Page number from the PDF and DOI link
- Weak support is flagged, never hidden

Extracted text you can check and correct



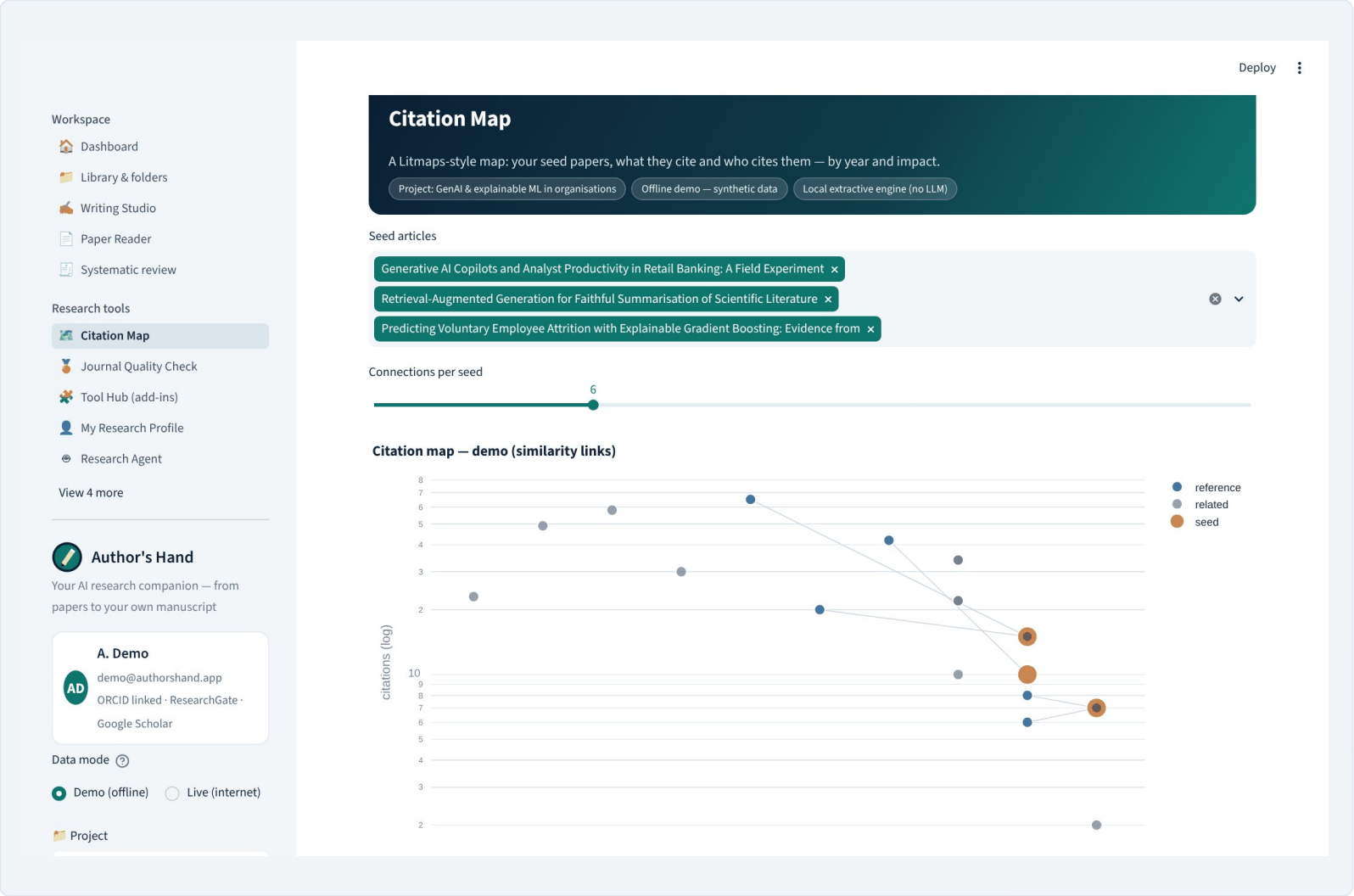
- Page-by-page text viewer
- Flags: empty / scanned pages, tables and figures mentioned
- Optional OCR for scanned PDFs
- Correct the text — summaries re-run on your version

Citation record with field provenance



- All 9 styles side by side
- Where each field came from (Crossref, PDF, edited by you) and when
- Edit any field; fill from DOI
- Notes and tags carried into BibTeX / RIS exports

Citation map (Litmaps-style)



- Seed papers, references and citing papers by year and impact
- Export as BibTeX to Litmaps, ResearchRabbit, Zotero

From summariser to a full research workspace

One identity

E-mail = workspace; ORCID (found from your e-mail), ResearchGate and Zotero linked once.

Projects & folders

Every project keeps its own folders, articles, citation style, add-ins and saved search.

Auto-refresh

New papers on your topic and papers citing your library arrive as alerts; citation counts update.

Writing Studio

Pick 5 articles → overall summary + Intro, LR, Gap, Method, Model, RQs, Hypotheses, Future study.

Every citation style

Editors, edition, pages, publisher kept; 9 built-in styles + 10,000 CSL styles; copy with italics.

Visual mode

Funnels, timelines, gap heat-maps, white-space maps, conceptual-model diagrams.

Journal quality check

DOAJ, OpenAlex, Crossref + your Scopus/ABDC/UGC-CARE lists + red flags → score.

45 add-ins

Litmaps, SciSpace, JANE, Connected Papers, Zotero, Elicit, scite … per writing stage.

Writing Studio: ten tabs, one for each part of the paper

1

Summaries

Overall + per-article summary, cheatsheet PDF

2

Introduction

Funnel + opening evidence

3

Literature review

Timeline, themes, comparison matrix

4

Research gap

Gap types, who-reports-what, white-space map

5

Methodology

Methods & designs charts, checklist

6

Model development

Conceptual model diagram

7

Research questions

RQ cards from gaps

8

Hypotheses

Hypothesis path diagram + support

9

Future study

Directions by type

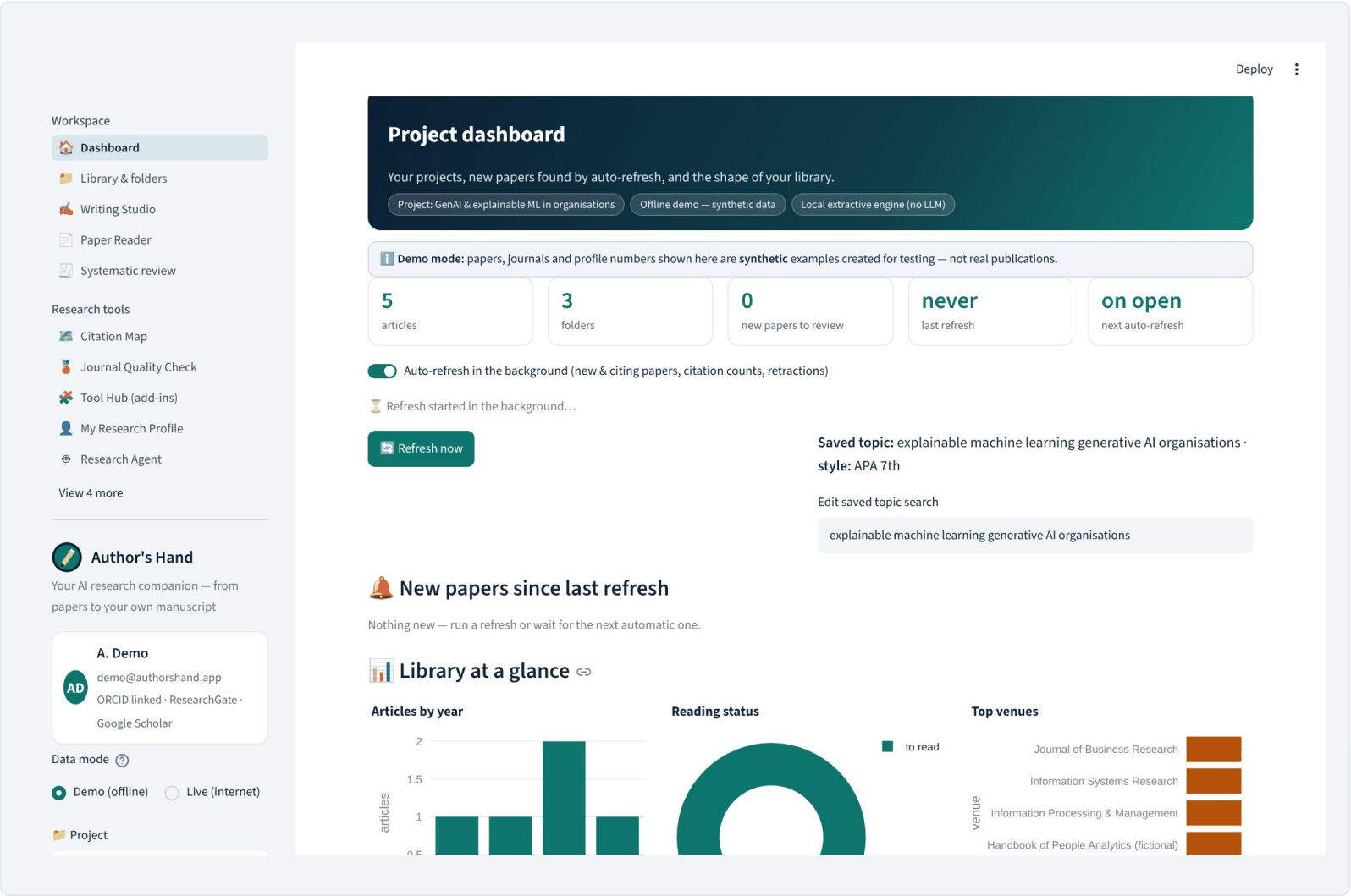
10

References

9 styles, Word/BibTeX/RIS/EndNote

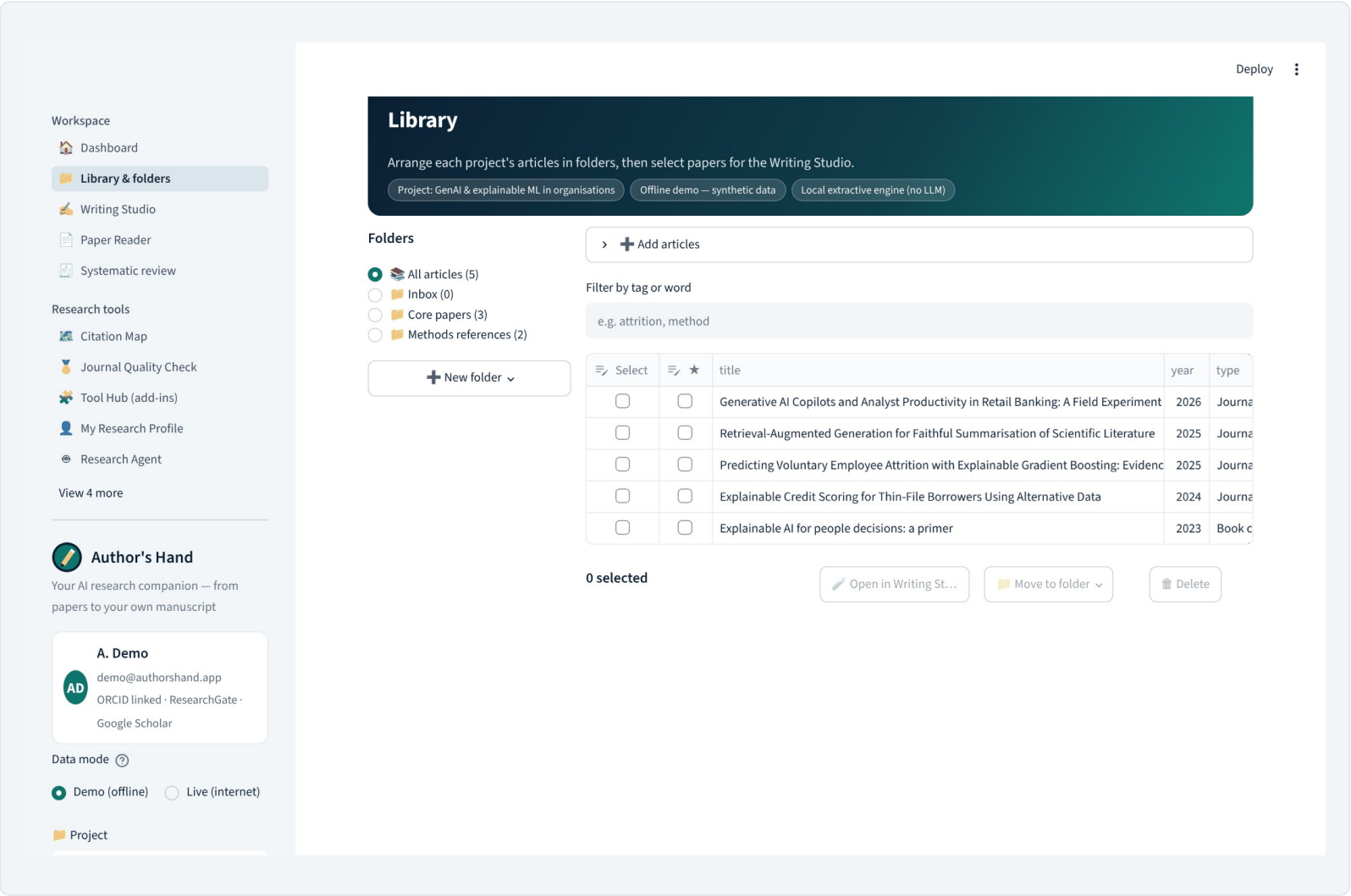
Every tab: Visual mode on/off · copy-ready text box (copy icon) · the add-ins that fit that stage · drafts always cited; local drafts must be paraphrased before submission.

Project dashboard with background auto-refresh



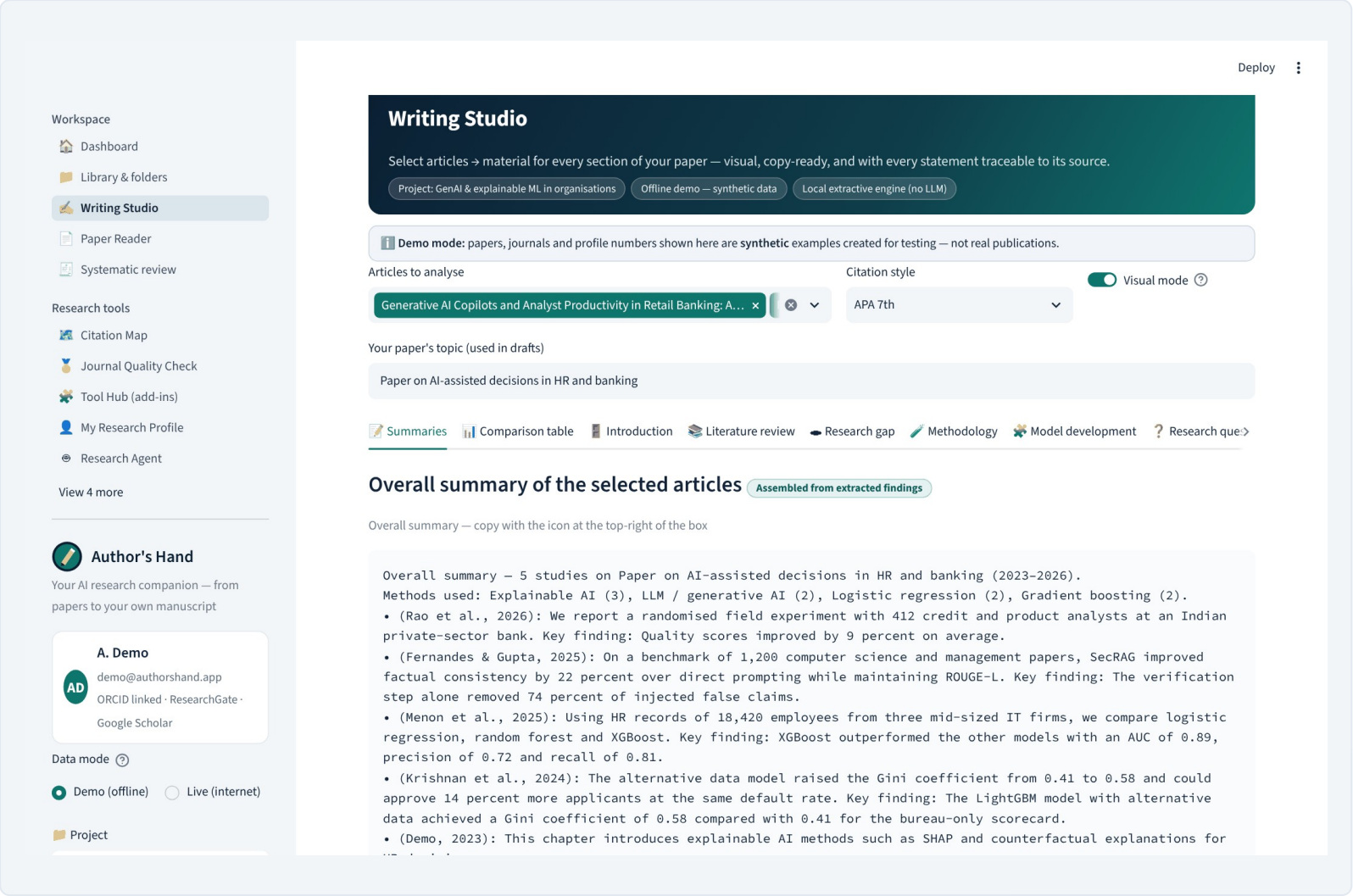
- Project KPIs, last and next refresh
- Refresh runs in the background — the app stays responsive
- New papers on the saved topic / citing your library → add to a folder
- Library charts: years, reading status, venues

Library: projects → folders → articles



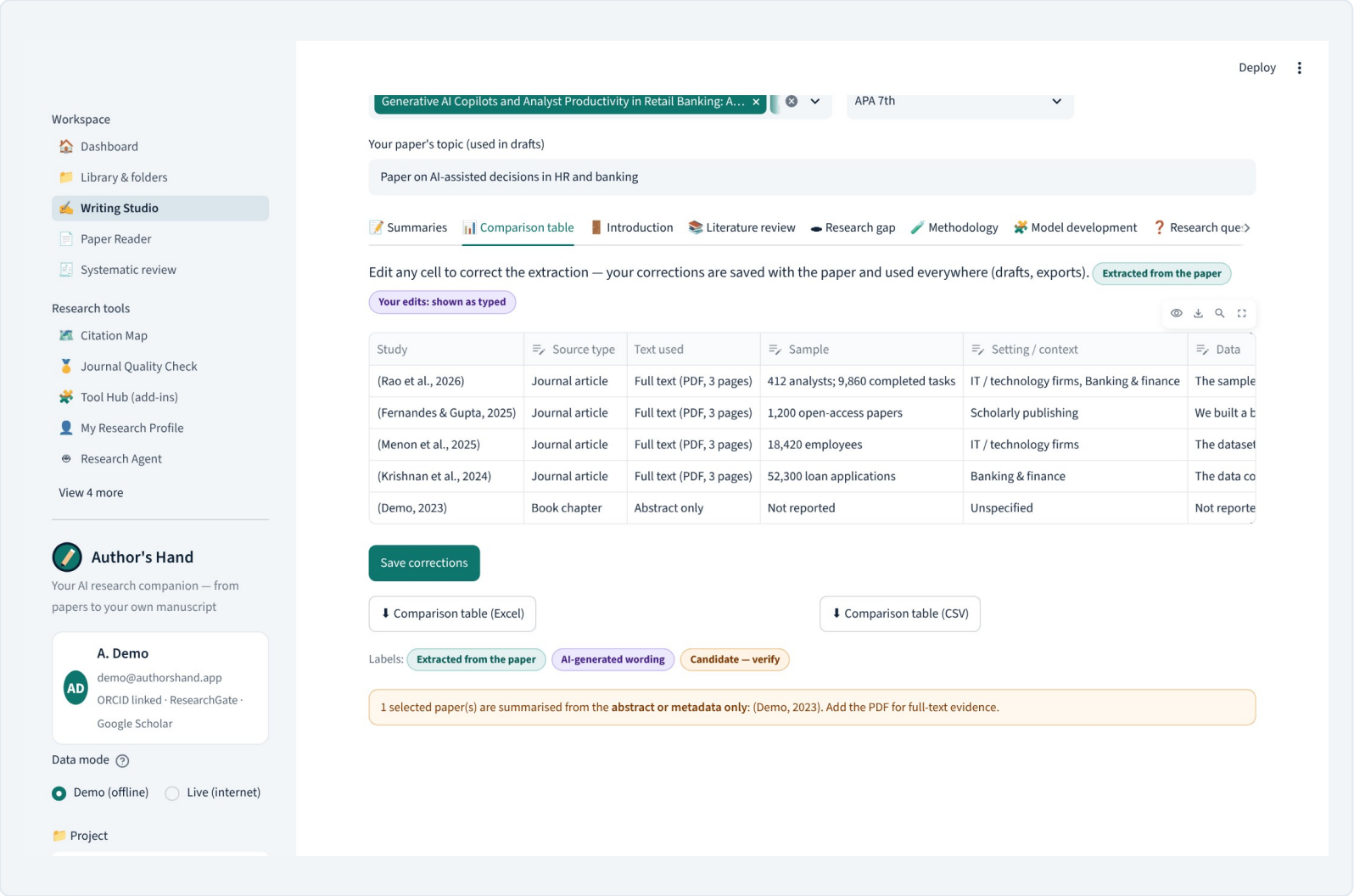
- Search, DOI/arXiv, PDF upload, BibTeX/RIS import (Litmaps, ResearchRabbit, Zotero, Mendeley, ResearchGate) or your ORCID works
- Type, text used, tags and notes columns (editable)
- Filter by tag or word

Overall summary of the selected articles — copy in one click



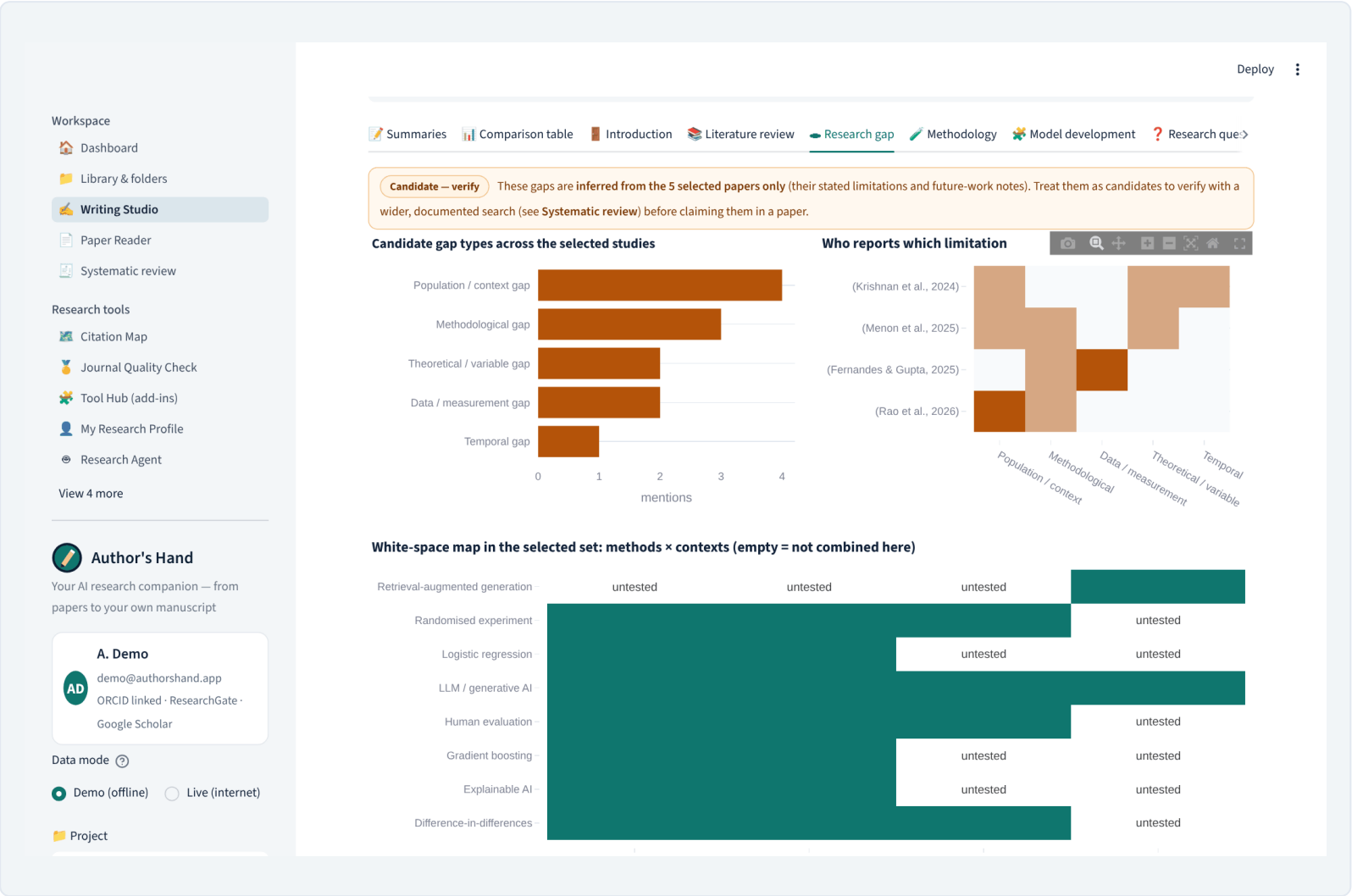
- One overall summary across all selected papers
- Badge: assembled from extracted findings
- Per-article cards, evidence and cheatsheets
- Warns when a paper is abstract-only

Comparison table you can correct and export



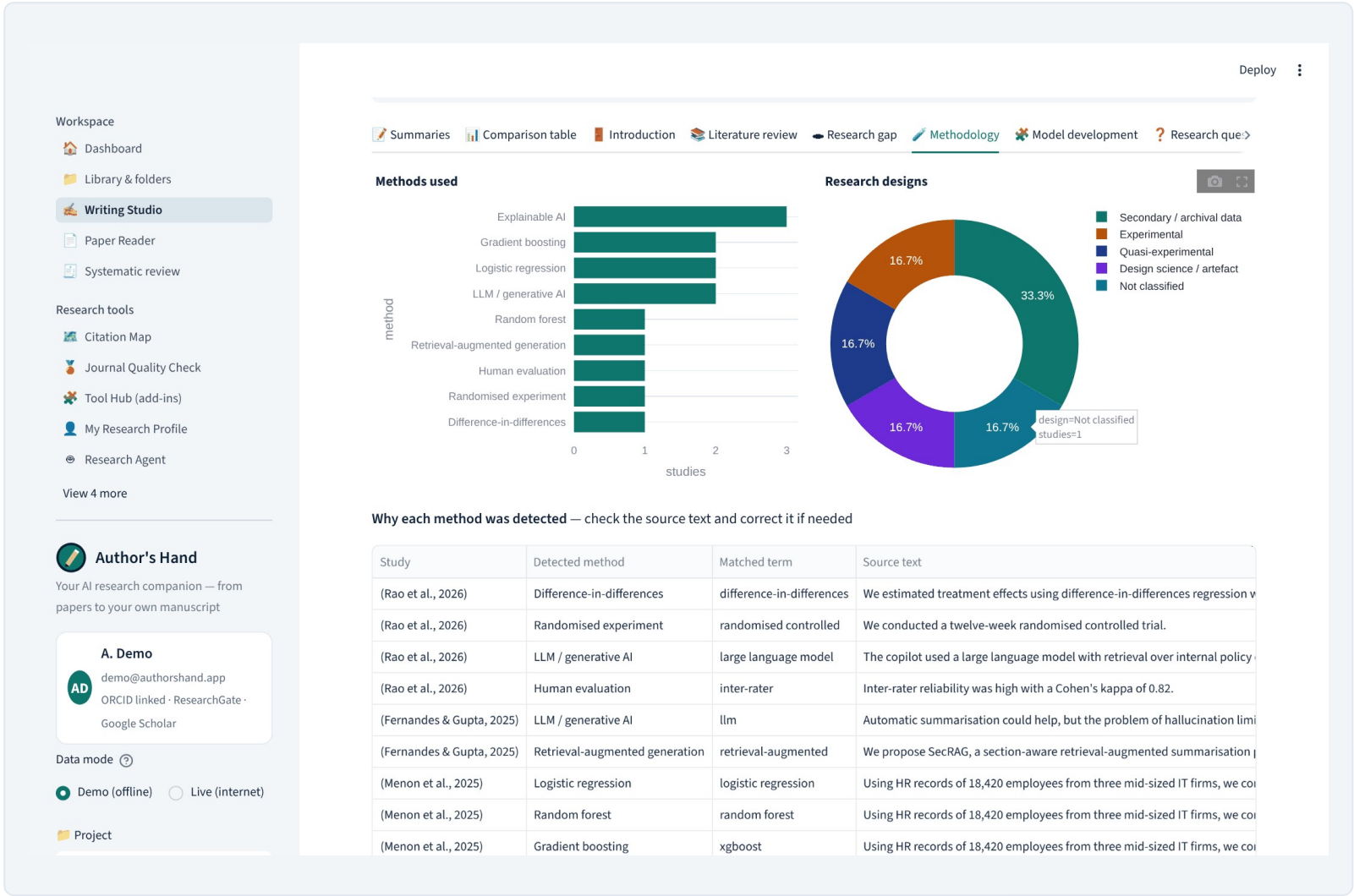
- Source type, text used, sample, setting, data, method
- Edit any cell → saved and reused in every draft
- Excel / CSV export

Research gap: candidates, clearly labelled



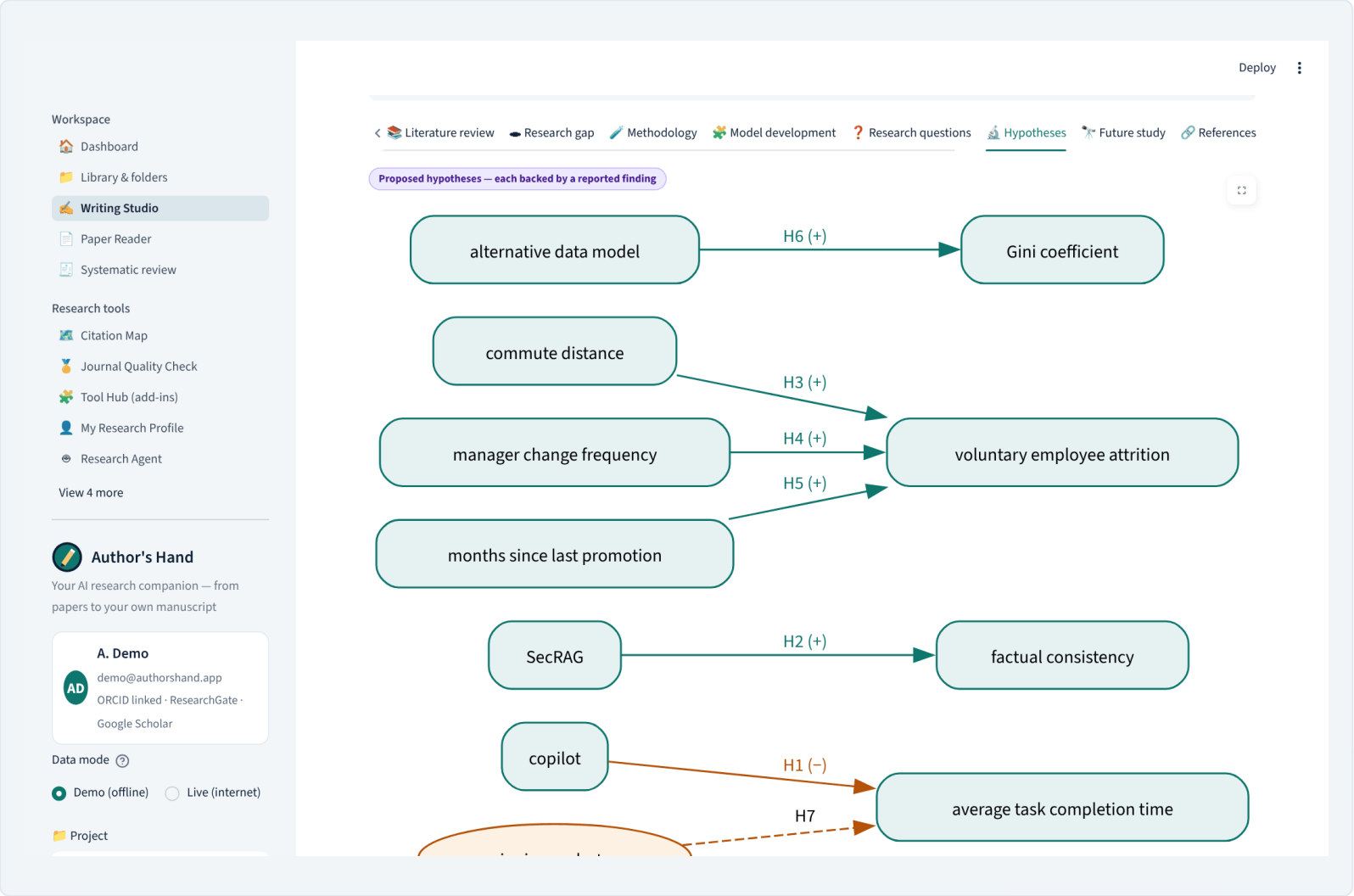
- “Candidate — verify” label on every gap
- Gap types across studies, who reports what
- White-space map: method × context combinations not studied

Methodology: why each method was detected



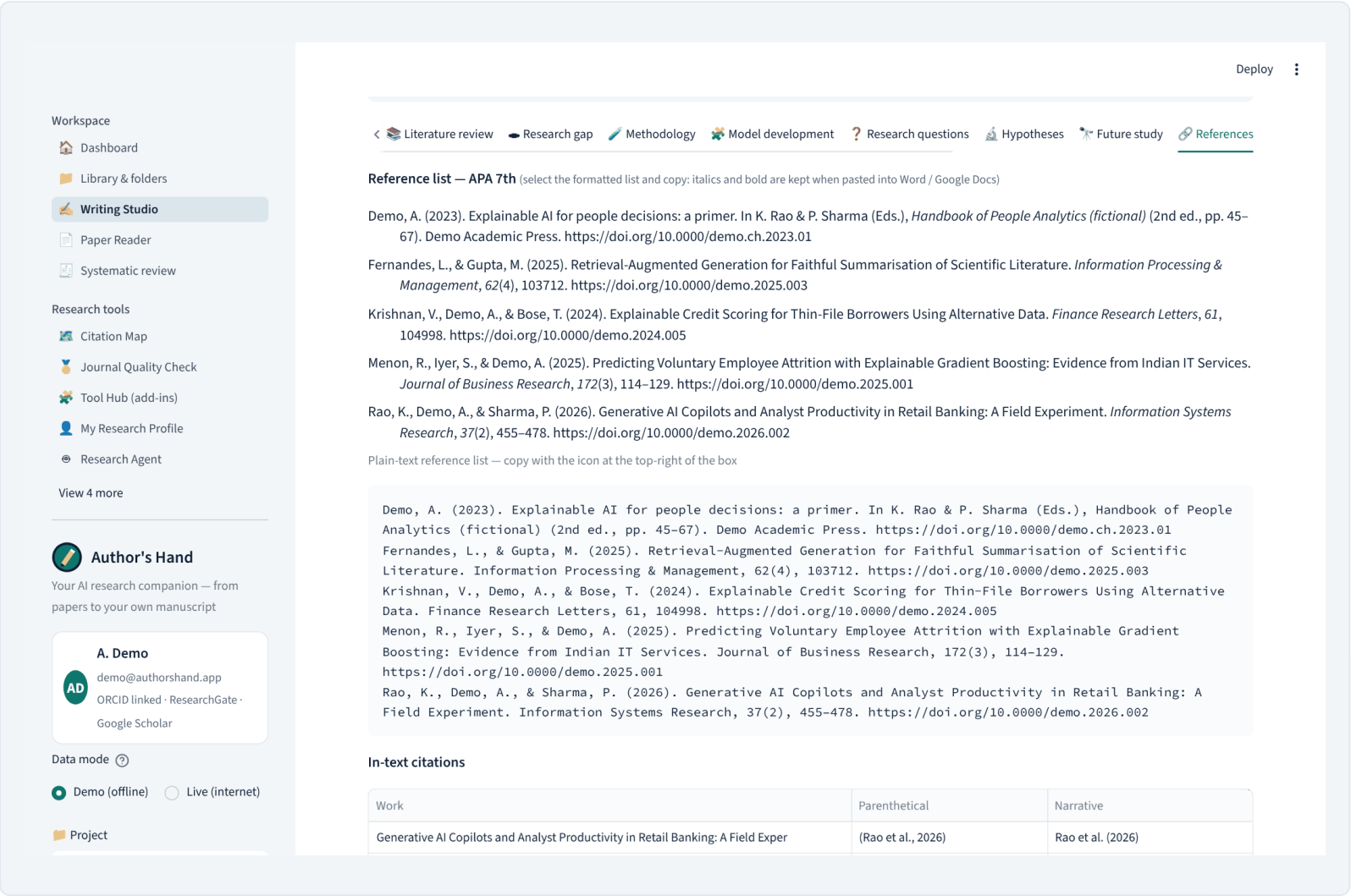
- Methods and designs charts
- Table: detected method, matched term, source sentence
- Correct the classification; add your field's own terms

Model development & hypotheses



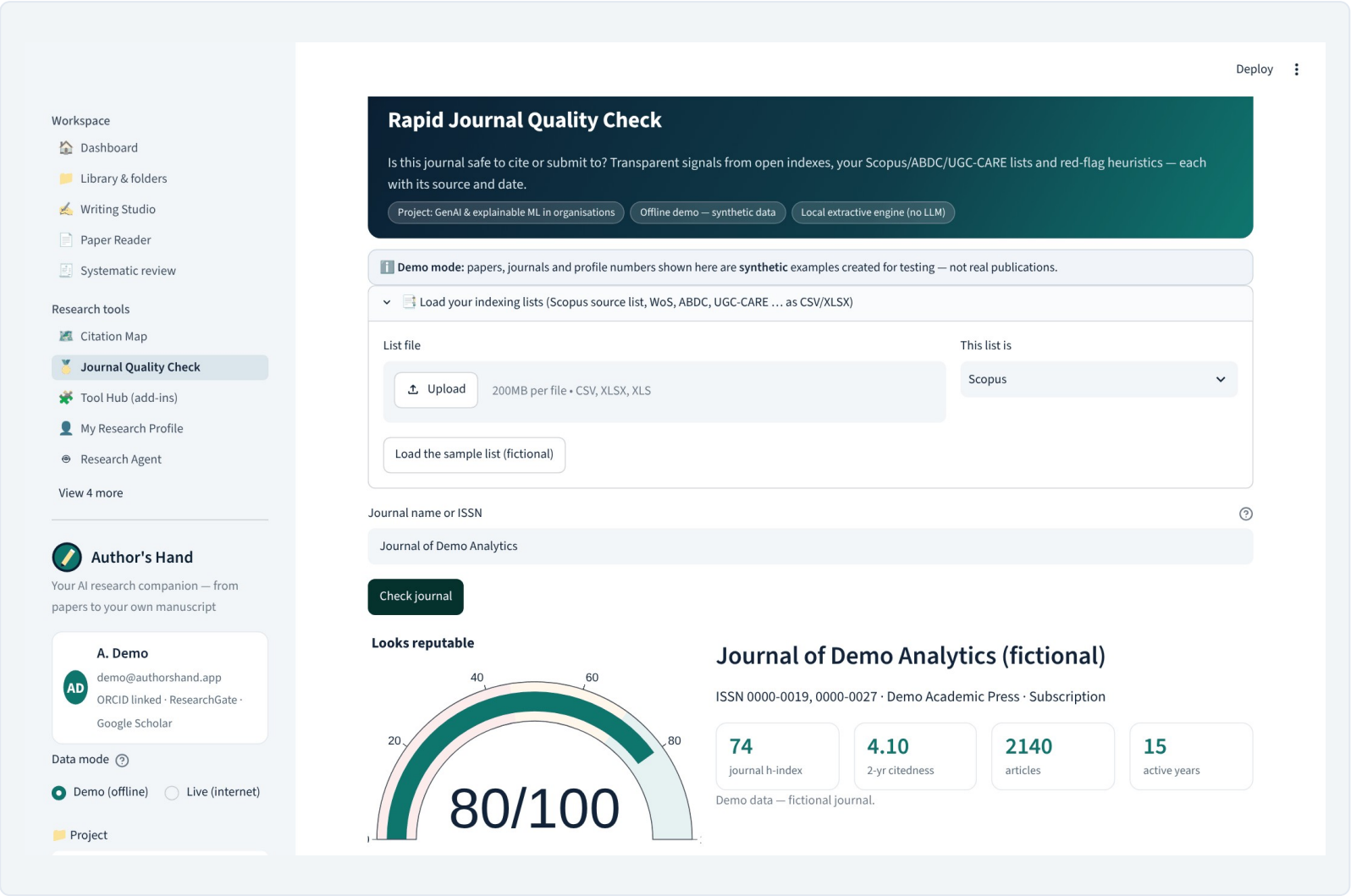
- Relationships extracted from findings (X → Y, sign)
- H1…Hn each backed by a cited finding
- Hidden automatically for review-only goals

References: full citations in the style you need



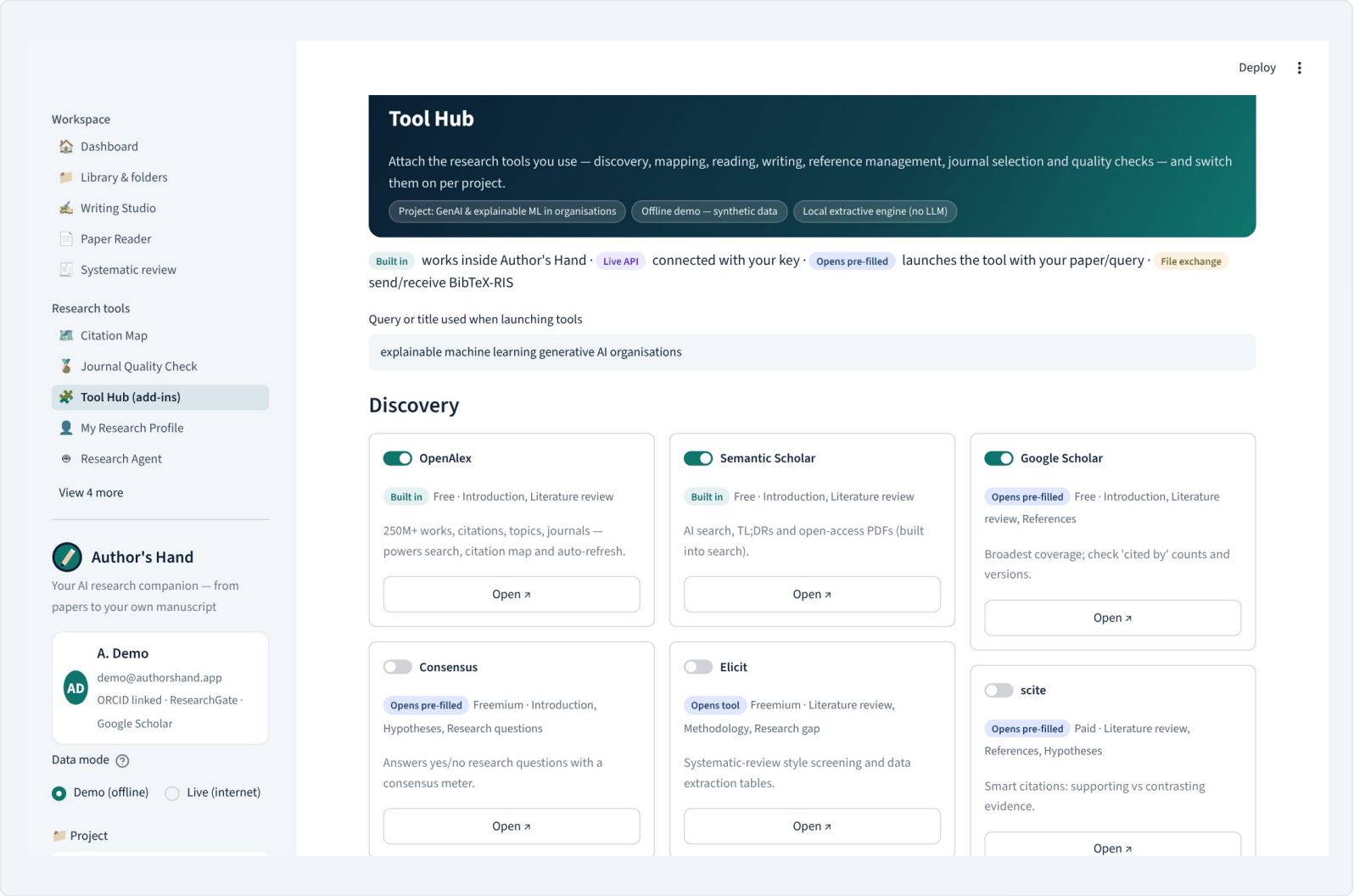
- Editors, edition, volume, issue, pages, publisher
- APA, MLA, Chicago ×2, Harvard, IEEE, Vancouver, AMA, Nature + 10,000 CSL styles via DOI
- Word (italics kept), BibTeX, RIS, EndNote

Rapid Journal Quality Check — a screening aid



- Signals from DOAJ, OpenAlex, Crossref and your own indexing lists
- Each signal shows its data source and the date checked
- Explained as a screening aid, not a universal quality score

Tool Hub: 45 add-ins, honestly labelled



- Built in · live API · opens pre-filled · file exchange
- JANE, SciSpace, Litmaps, Elicit, scite, Zotero …
- Enabled add-ins appear on the matching Writing Studio tab

Cheatsheet, Windows app and Play Store



PAPER CHEATSHEET · AUTHOR'S HAND

Predicting Voluntary Employee Attrition with Explainable Gradient Boosting: Evidence from Indian IT Services

R. Menon, S. Iyer, A. Demo · 2025 · *Journal of Business Research*

TL;DR — Using HR records of 18,420 employees from three mid-sized IT firms, we compare logistic regression, random forest and XGBoost.

0.89	18,420	Secondary / archival data	Machine-learning prediction
AUC	employees	research design	approach

RESEARCH PROBLEM
- However, most predictive attrition studies rely on the public IBM HR dataset, which is synthetic and small.
- Little is known about how well attrition models generalise to real Indian workforce data.

KEY FINDINGS
- XGBoost outperformed the other models with an AUC of 0.89, precision of 0.72 and recall of 0.81.
- SHAP analysis revealed that months since last promotion was the single most important driver, followed by commute distance above 25 kilometres and more than two manager changes in a year.
- The fairness audit found a difference of only 1.8 percentage points in false positive rates between men and women.

OBJECTIVES
- This study addresses both gaps.
- Our objective is to build an accurate and explainable early-warning model and to examine which organisational drivers matter most.

CONTRIBUTIONS
- HR managers can use the individual SHAP explanations to plan targeted interventions such as internal mobility offers and stable reporting lines.
- In addition, black-box predictions are rarely adopted by HR managers because they cannot explain why an employee is at risk.

METHODOLOGY
- Three models were trained: logistic regression, random forest and XGBoost.
- Voluntary attrition costs Indian IT services firms an estimated 1.5 to 2 times an employee's annual salary per exit.

LIMITATIONS
- The data come from only three firms in one industry, which limits generalisability.
- The model does not capture informal factors such as team climate or personal circumstances.

DATA & SAMPLE
- The dataset contains 18,420 employee records collected between 2019 and 2023 from three IT services firms in Bengaluru, Hyderabad and Pune.
- The voluntary attrition rate in the sample is 17.6 percent.

FUTURE RESEARCH
- Future research should test the model in other sectors, incorporate engagement survey text using natural language processing, and evaluate the causal effect of the recommended interventions through field experiments.

CITE (APA)

Menon, R., Iyer, S., & Demo, A. (2025). Predicting Voluntary Employee Attrition with Explainable Gradient Boosting: Evidence from Indian IT Services. *Journal of Business Research*, *172*(3), 114–129. https://doi.org/10.0000/demo.2025.001

Keywords: attrition, voluntary, model, explainable, indian, services, hr, firms · Methods: Logistic regression, Random forest, Gradient boosting, Explainable AI · Evidence support 1.00

Generated 29 Oct 2026 by Author's Hand · verify against the original paper before citing

Install anywhere

1. **Deploy**
   Streamlit Community Cloud (free) or Docker on Render / Cloud Run → public URL.

2. **Add to Home screen**
   Any phone: open the URL → browser menu → Install app.

3. **Play Store**
   mobile/ Capacitor wrapper → signed .aab in Android Studio → Google Play Console (US$25 once).

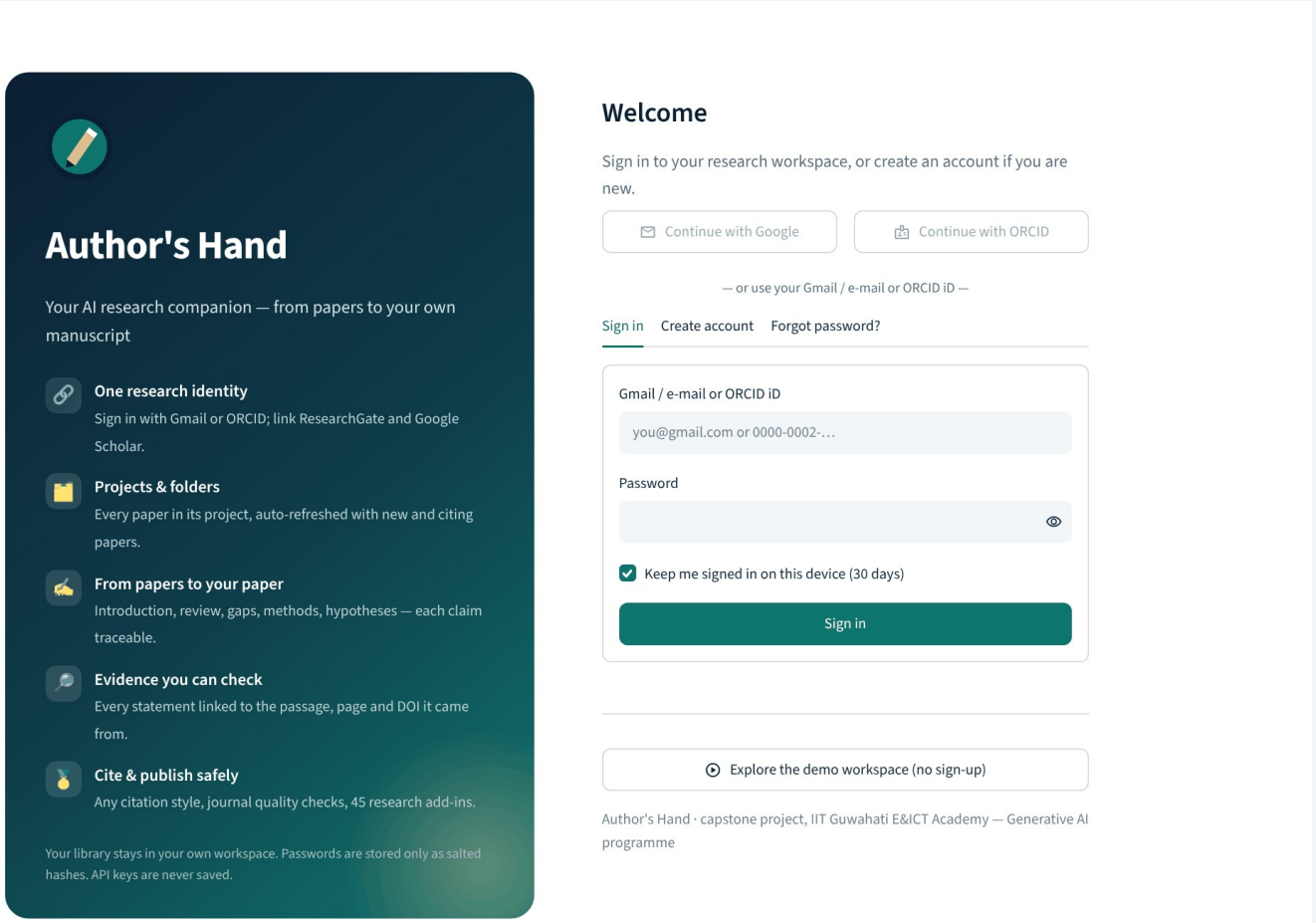
4. **Windows desktop app**
   AuthorsHand.exe — first run installs everything automatically; opens in its own window with a Start-menu shortcut.

One-page cheatsheet per paper (left): headline numbers, design, approach, eight summary boxes and the formatted citation.

Version 3: review feedback → what changed

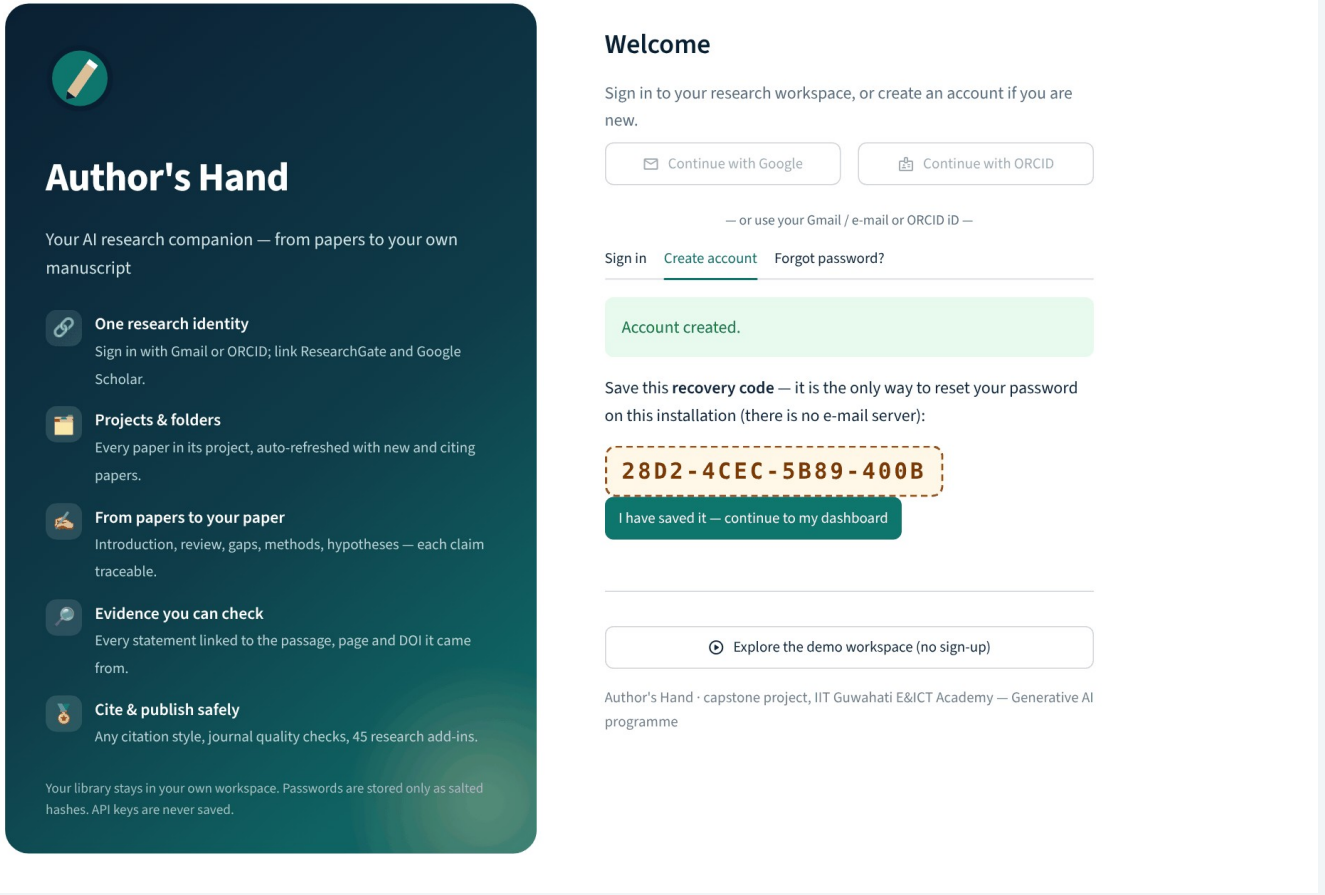
Requested (user · ChatGPT review · IIT G brief)	Delivered in Author's Hand v3
Login page; Gmail or ORCID iD; sign-up for new users	Login/sign-up page, recovery code, Continue with Google / ORCID (OpenID Connect), guided onboarding
Google Scholar profile with numbers and graph	Citations, h-index, i10 (all / since) and citations-per-year chart, with source and date
Trace every statement to its evidence	Click a statement → passage, context, section, page, DOI, support strength
Separate evidence from suggestions	Labels: Extracted from the paper · AI-generated wording · Candidate — verify
Reproducible literature review	Logged searches, duplicate reasons, editable screening, PRISMA 2020, methods paragraph, exports
PDF handling & corrections	Full text vs abstract label, page viewer, parse flags, OCR option, editable text, table and records
Privacy & control	What is stored/sent table, keys in memory only, unlink accounts, backup / restore / delete
Honest evaluation	Synthetic limitation stated; real-paper protocol, human ratings and failure cases built in

Login page: sign in, create account, or explore the demo



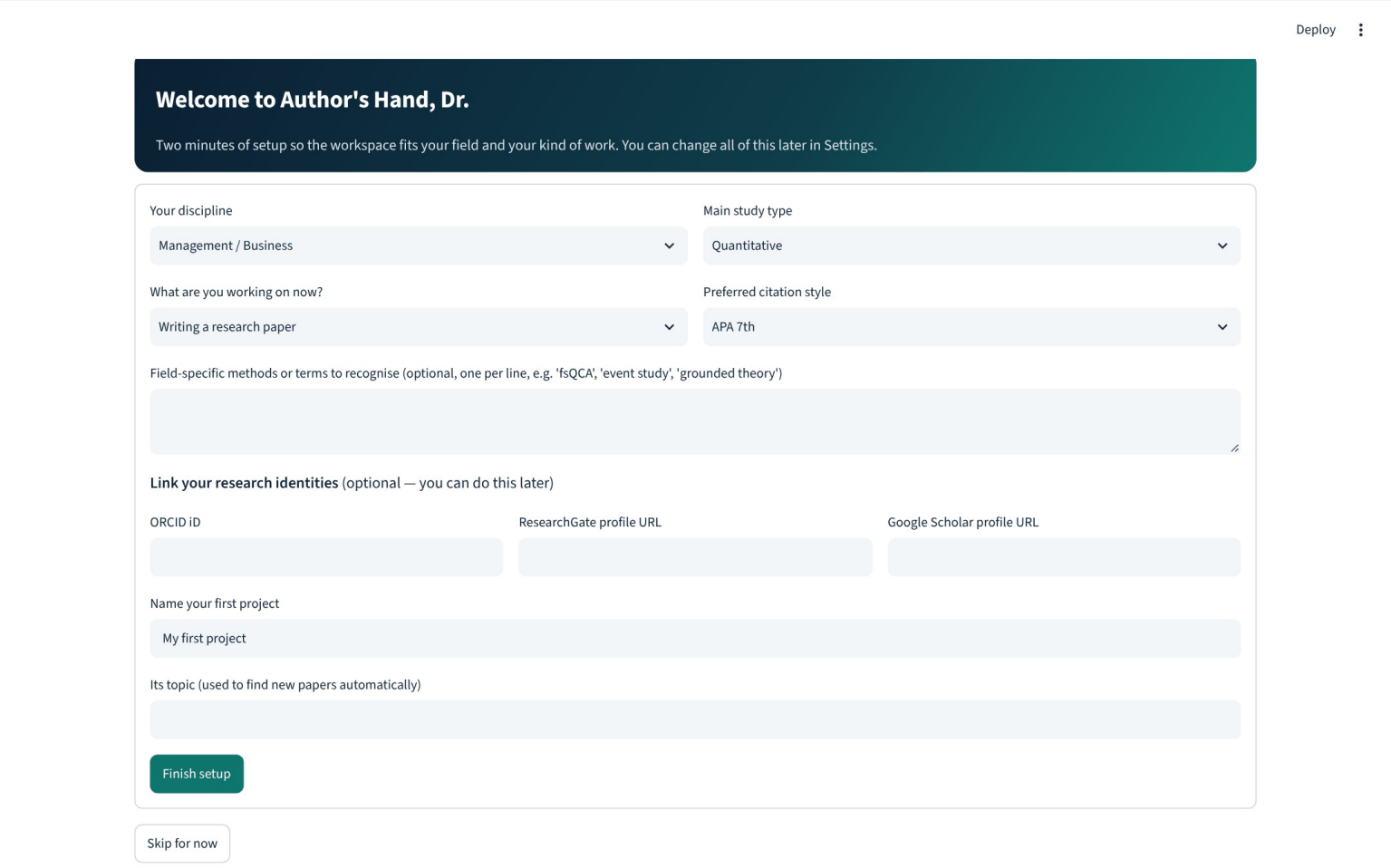
- Gmail / e-mail or ORCID iD + password
- Continue with Google / ORCID once client IDs are added (Settings → Sign-in providers)
- Keep me signed in (30 days, hashed token)
- Demo workspace without sign-up

First-time users: account + recovery code



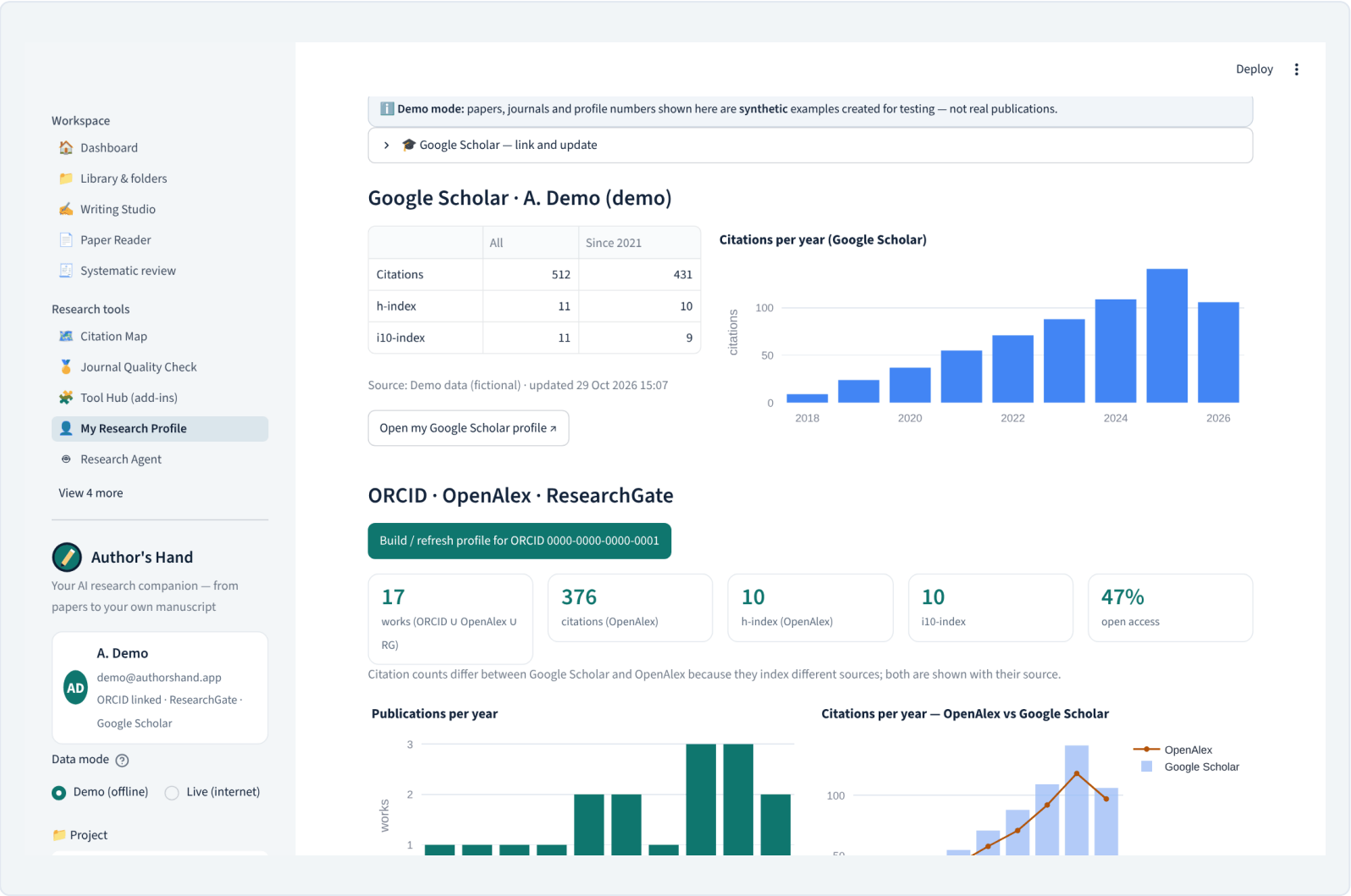
- Name, Gmail, optional ORCID iD, password twice, privacy consent
- Passwords stored only as salted PBKDF2-SHA256 hashes
- One-time recovery code replaces e-mail reset
- Then straight to guided setup

Guided setup: discipline, study type, goal, identities



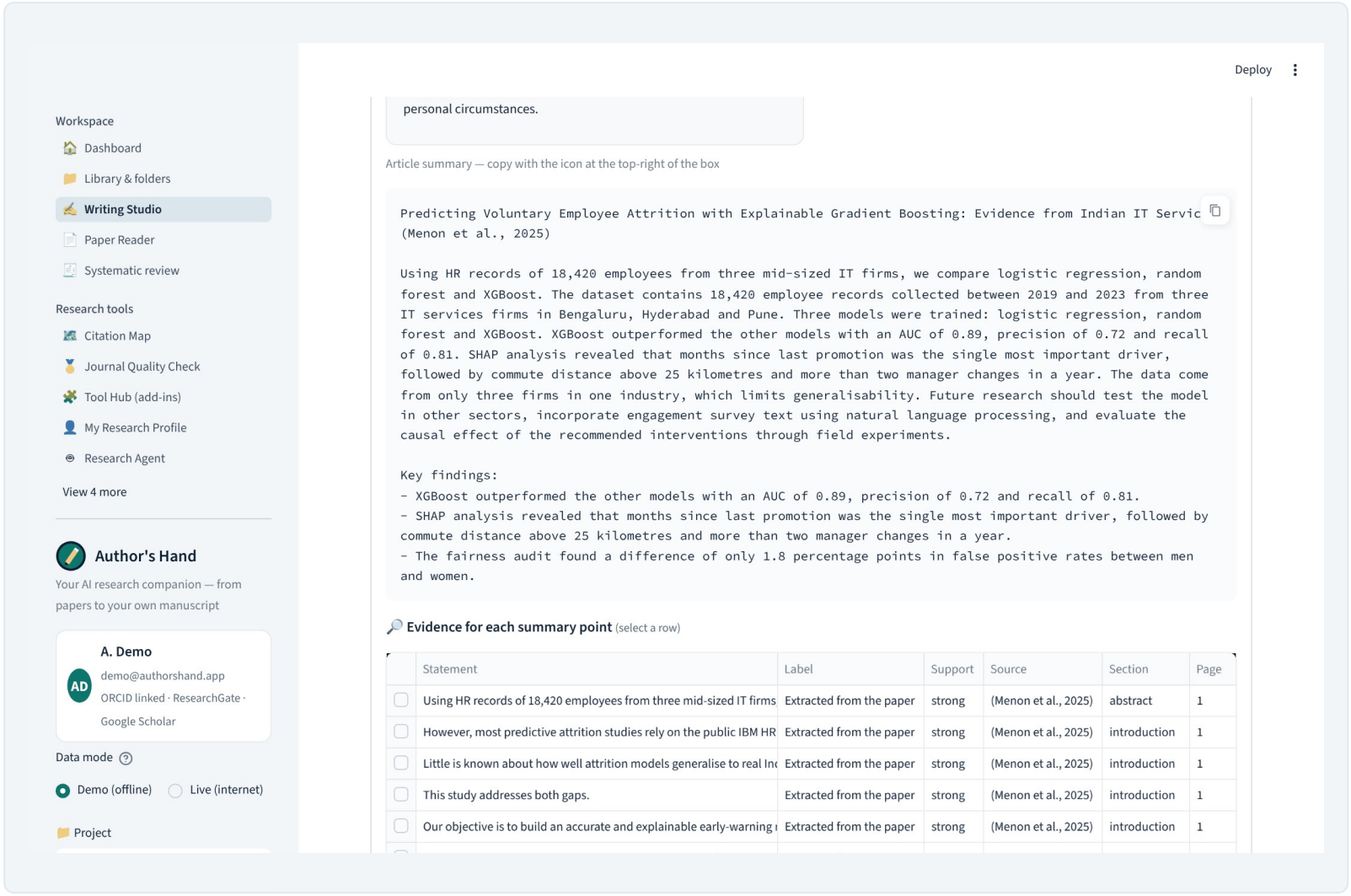
- Discipline, study type, current goal and citation style
- Field-specific method terms
- ORCID, ResearchGate and Google Scholar links (optional)
- First project created → personal dashboard

Google Scholar metrics and citation graph



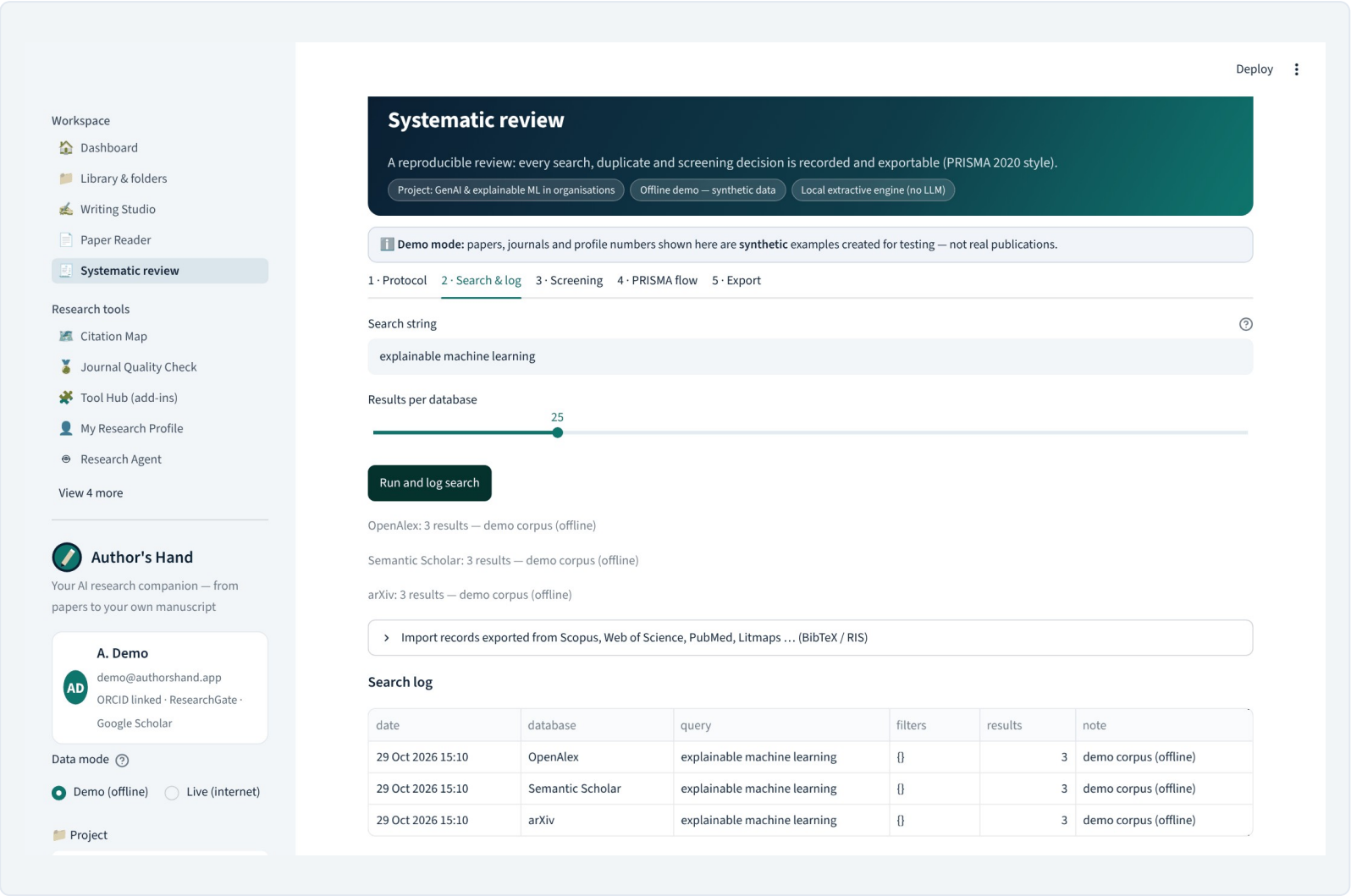
- Citations, h-index, i10 — all and since year
- Citations per year chart
- Three routes: SerpApi (own key), enter numbers, import export
- Source and date always shown; no scraping

Writing Studio evidence: statements you can check



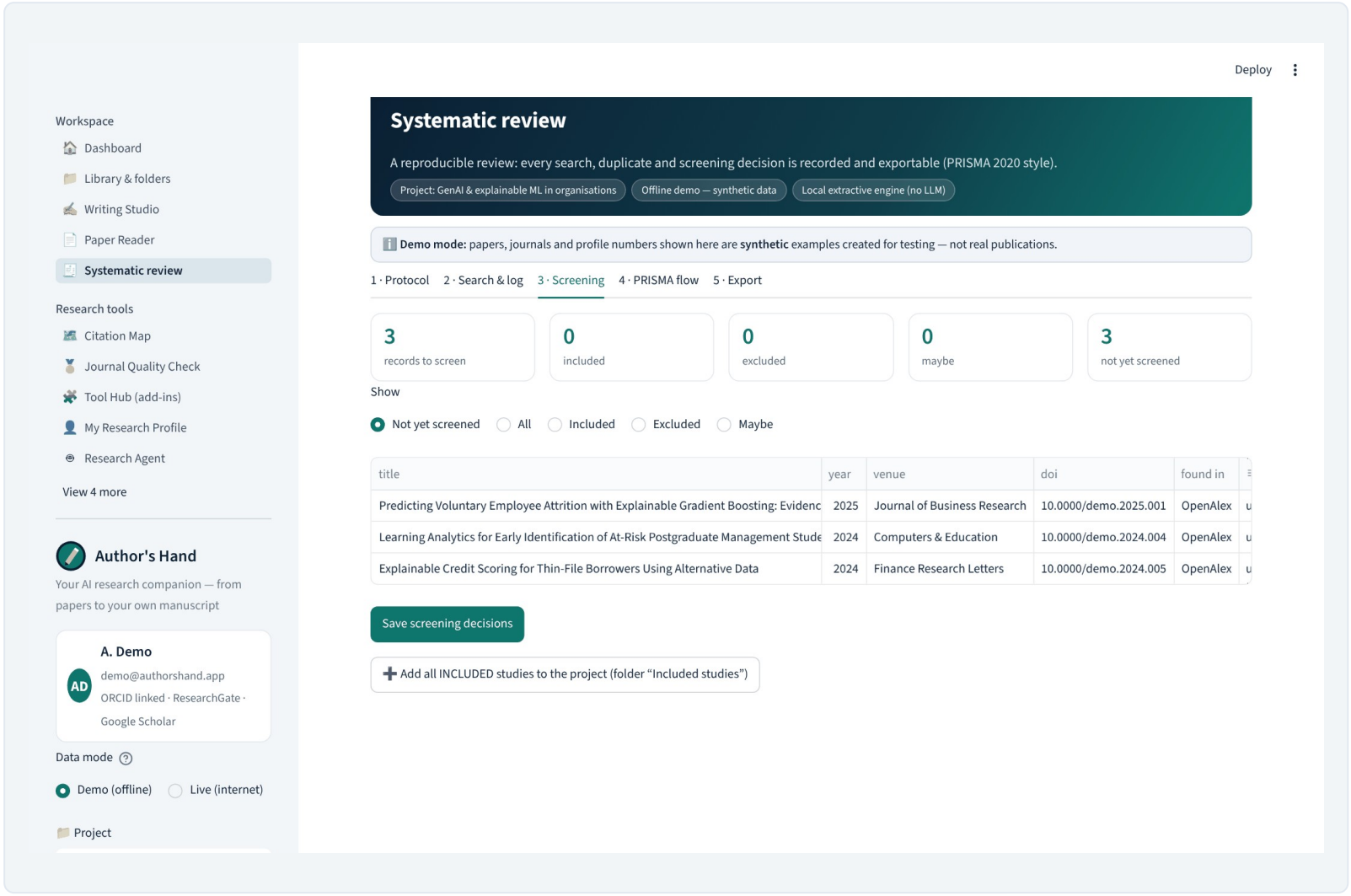
- Every summary point in a selectable table
- Label, support strength, source paper, section, page
- Detail card with passage, context and DOI link

Systematic review: every search logged



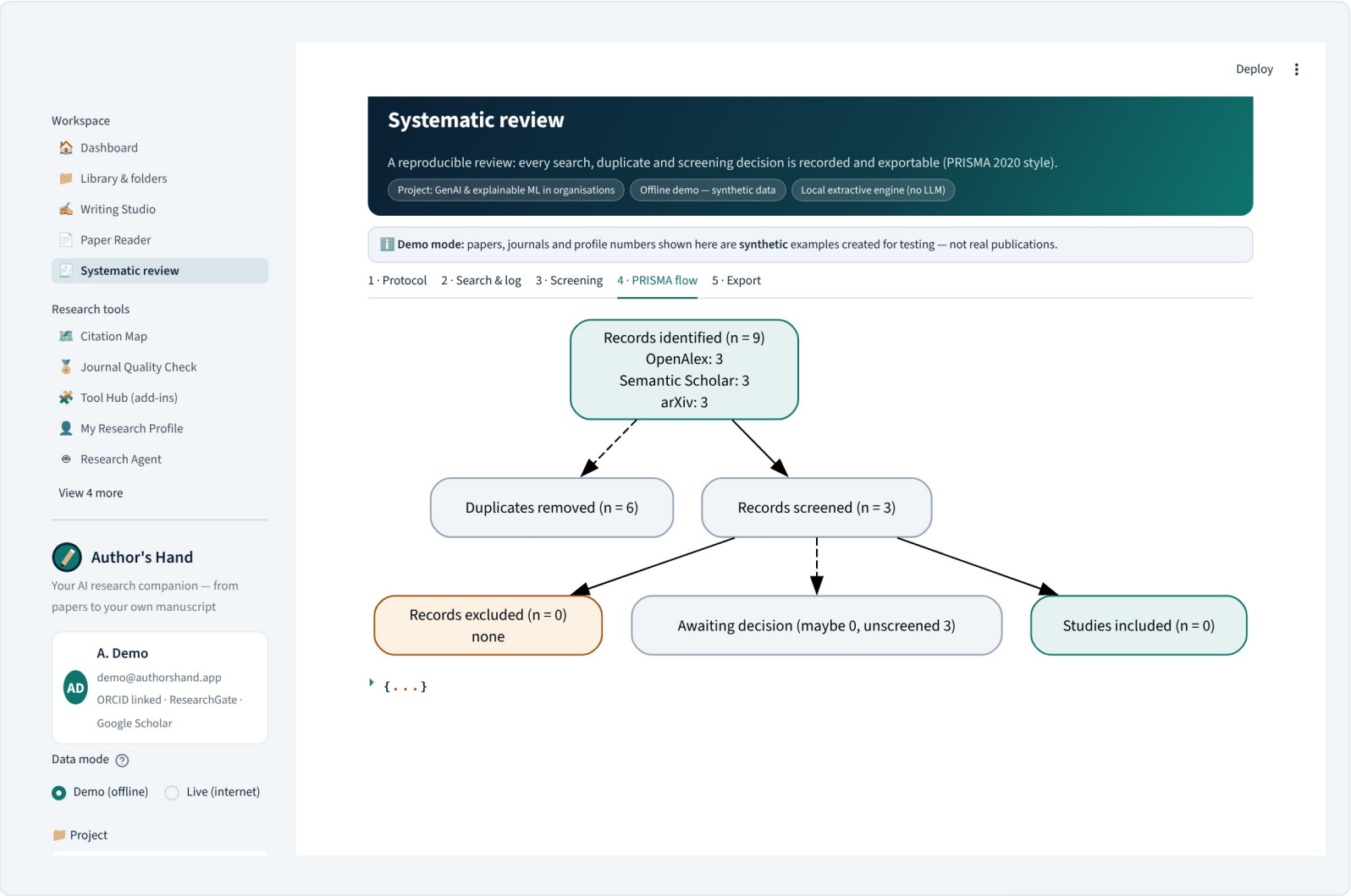
- Protocol: review type, question, inclusion & exclusion reasons
- Search string, databases, date, filters, results per database
- Import Scopus / WoS / PubMed exports as logged searches

Screening with decisions and reasons



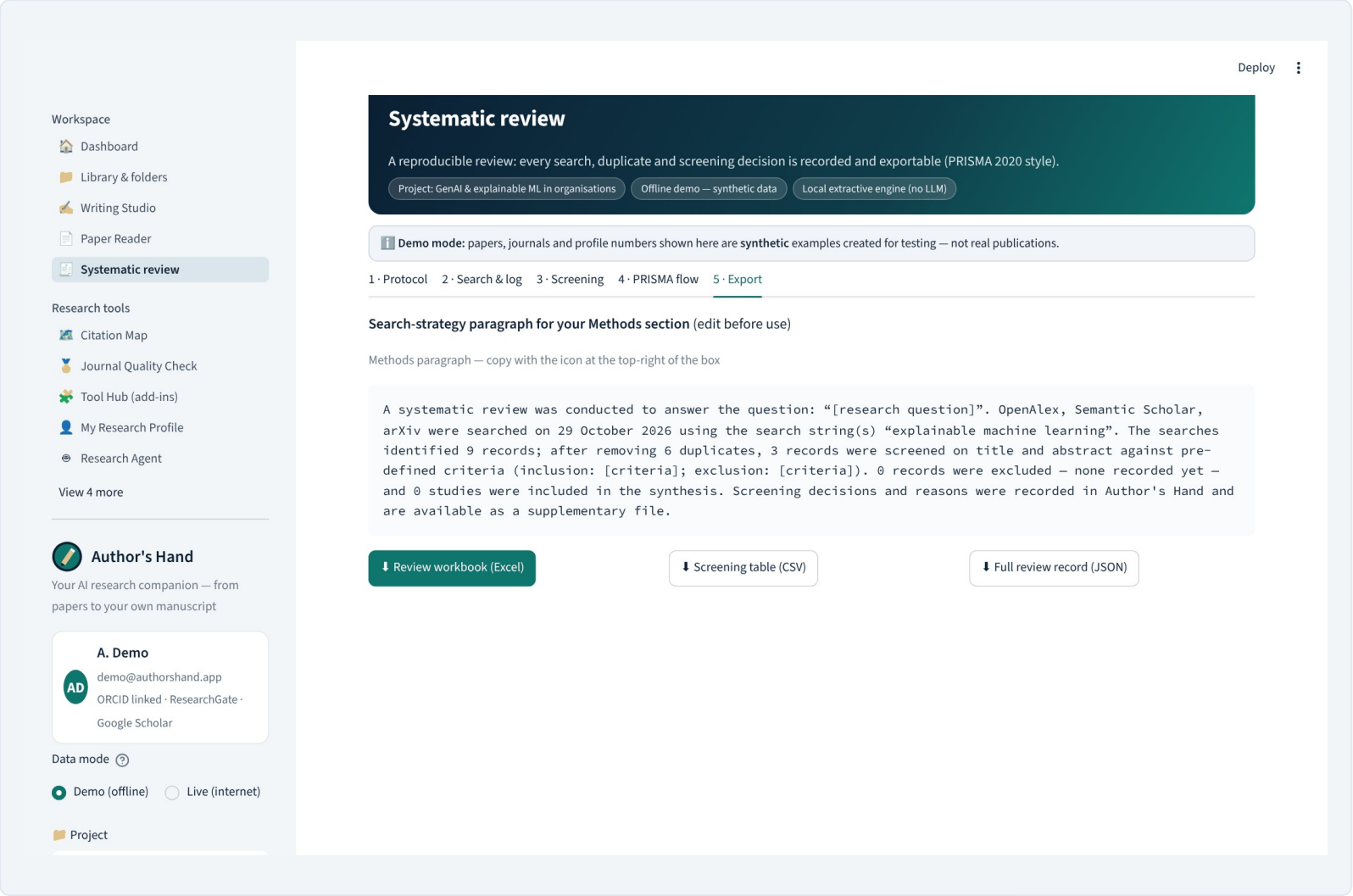
- Include / exclude / maybe per record
- Exclusion reason chosen from your protocol
- Included studies move to a project folder

PRISMA 2020 flow — generated from the log



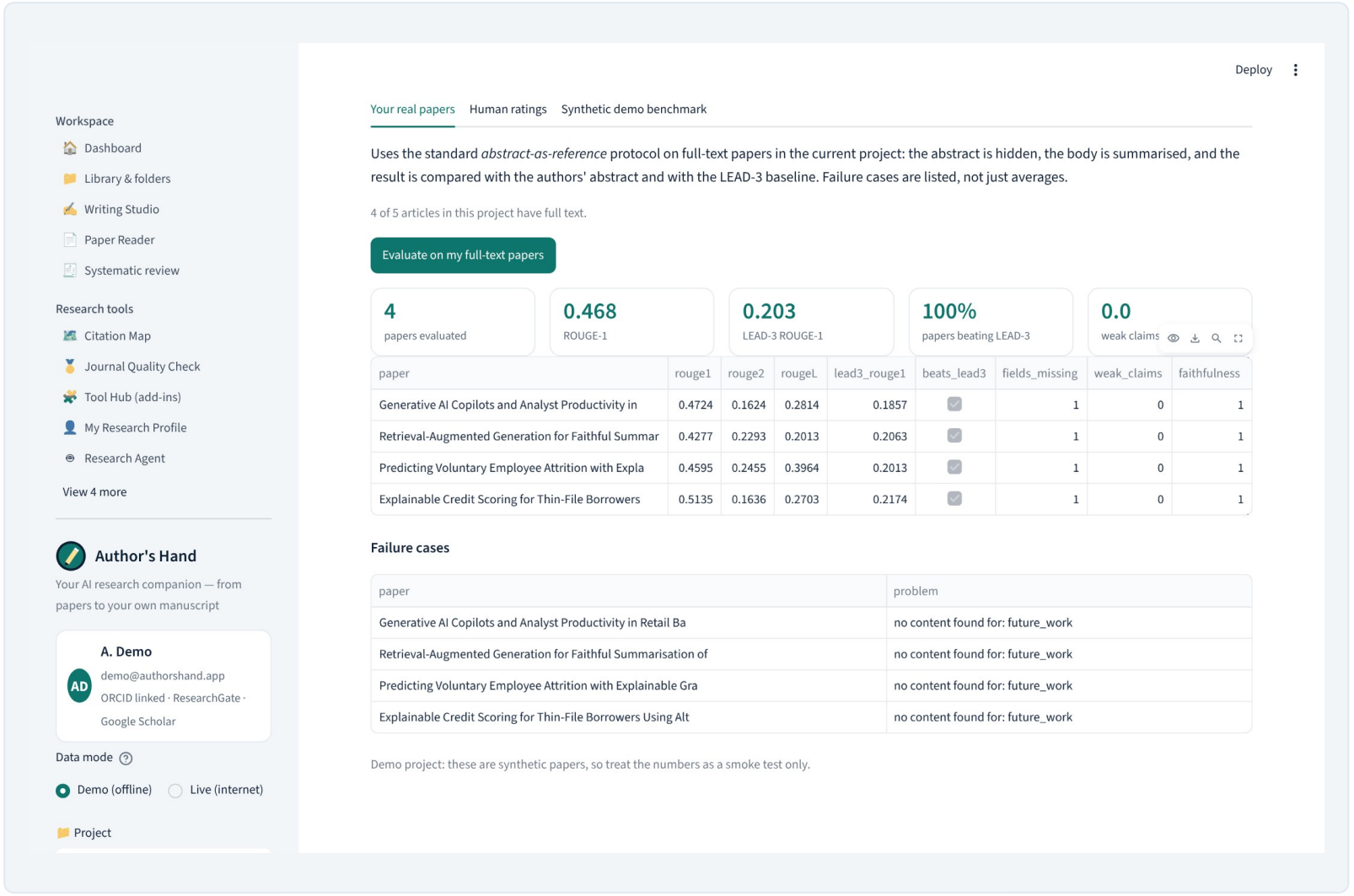
- Records per database, duplicates removed with reasons
- Screened, excluded (by reason), included
- Updates as you screen

Review export: methods paragraph and full record



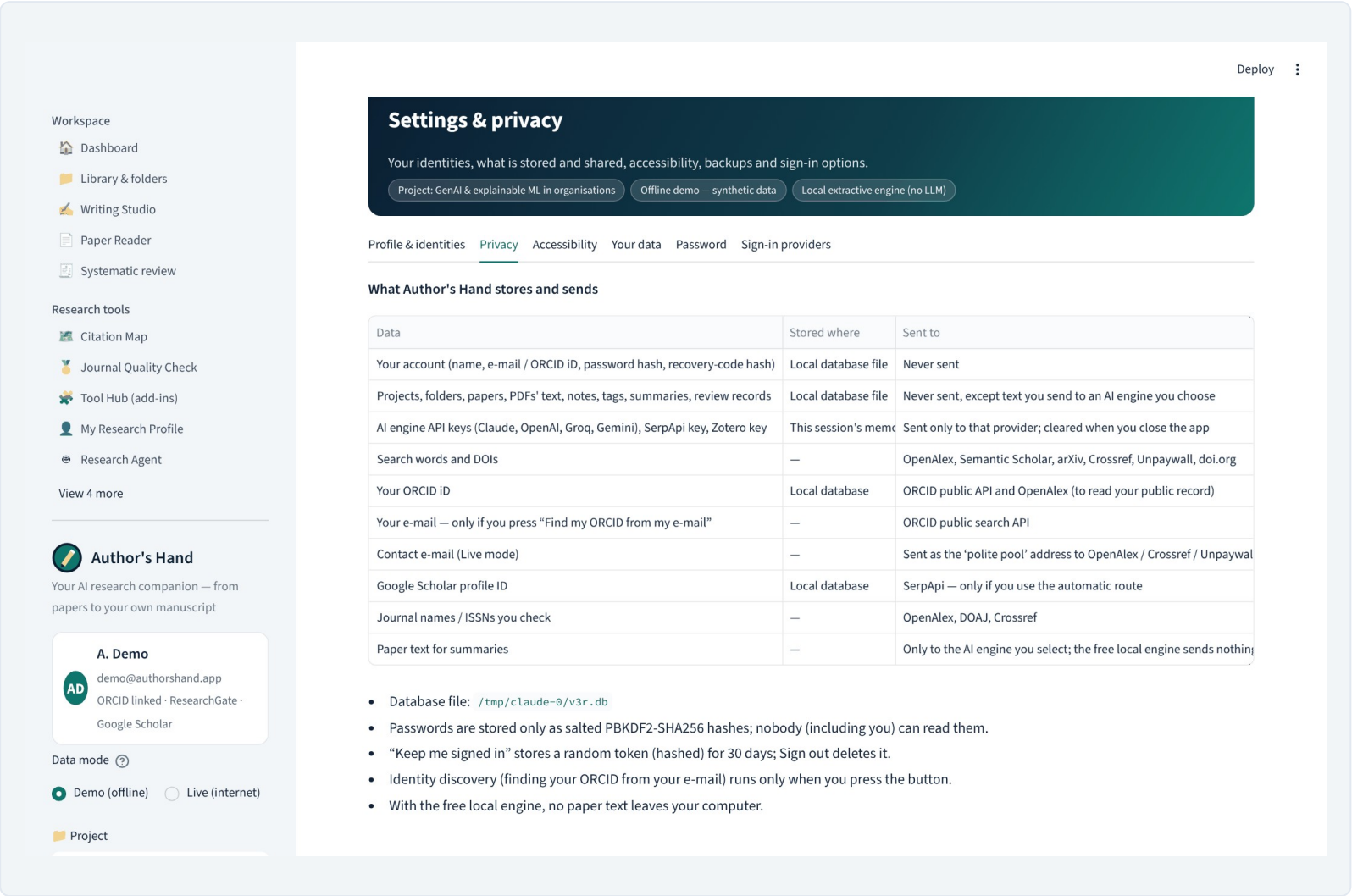
- Copy-ready search-strategy paragraph
- Excel workbook, screening CSV and JSON record
- Supplementary file for journals

Evaluation on your own real papers



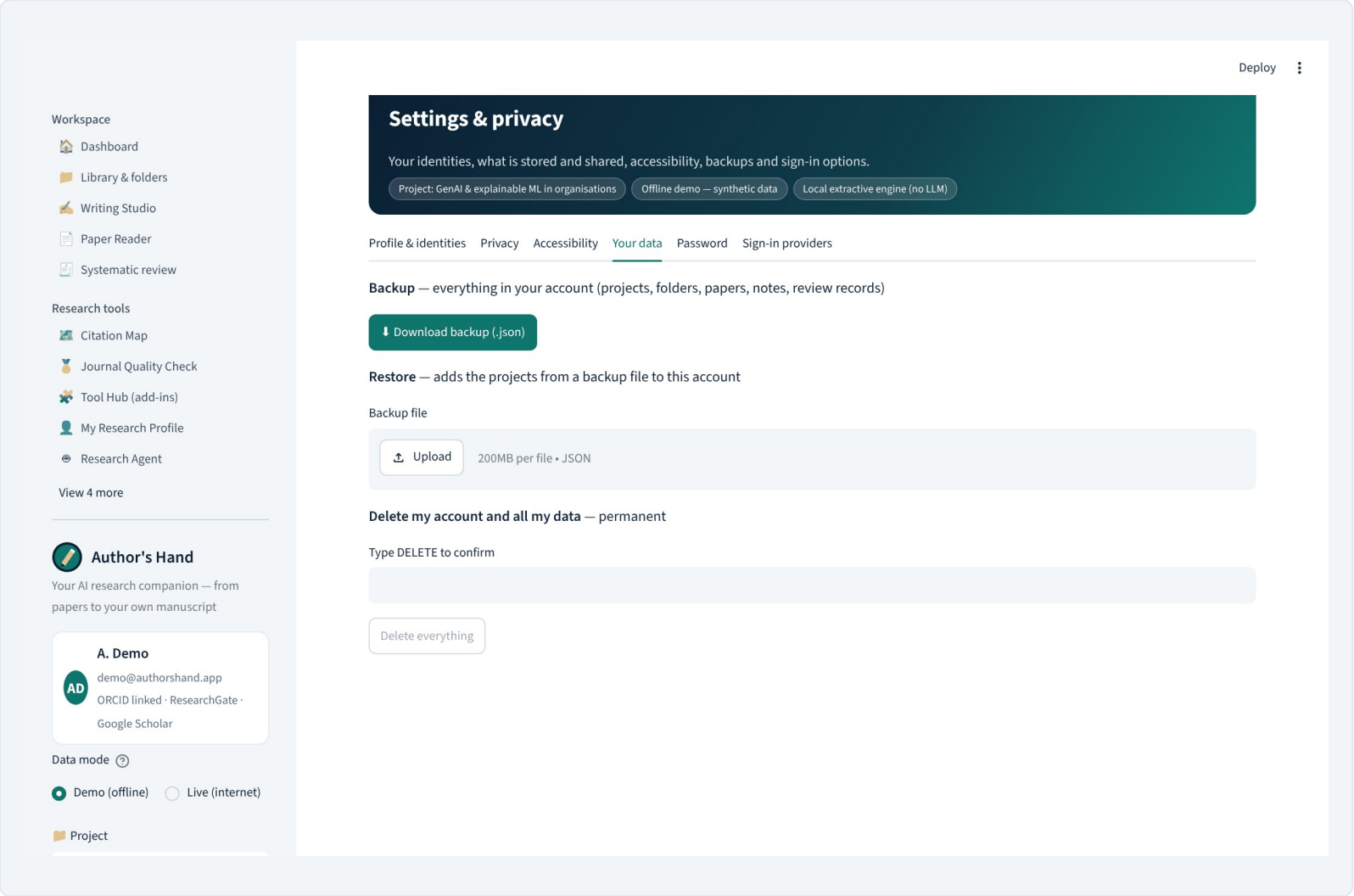
- Abstract hidden, body summarised, compared with the authors' abstract
- LEAD-3 baseline per paper
- Failure cases listed, not hidden
- Human ratings tab: accuracy, usefulness, errors

Privacy: what is stored and what is sent



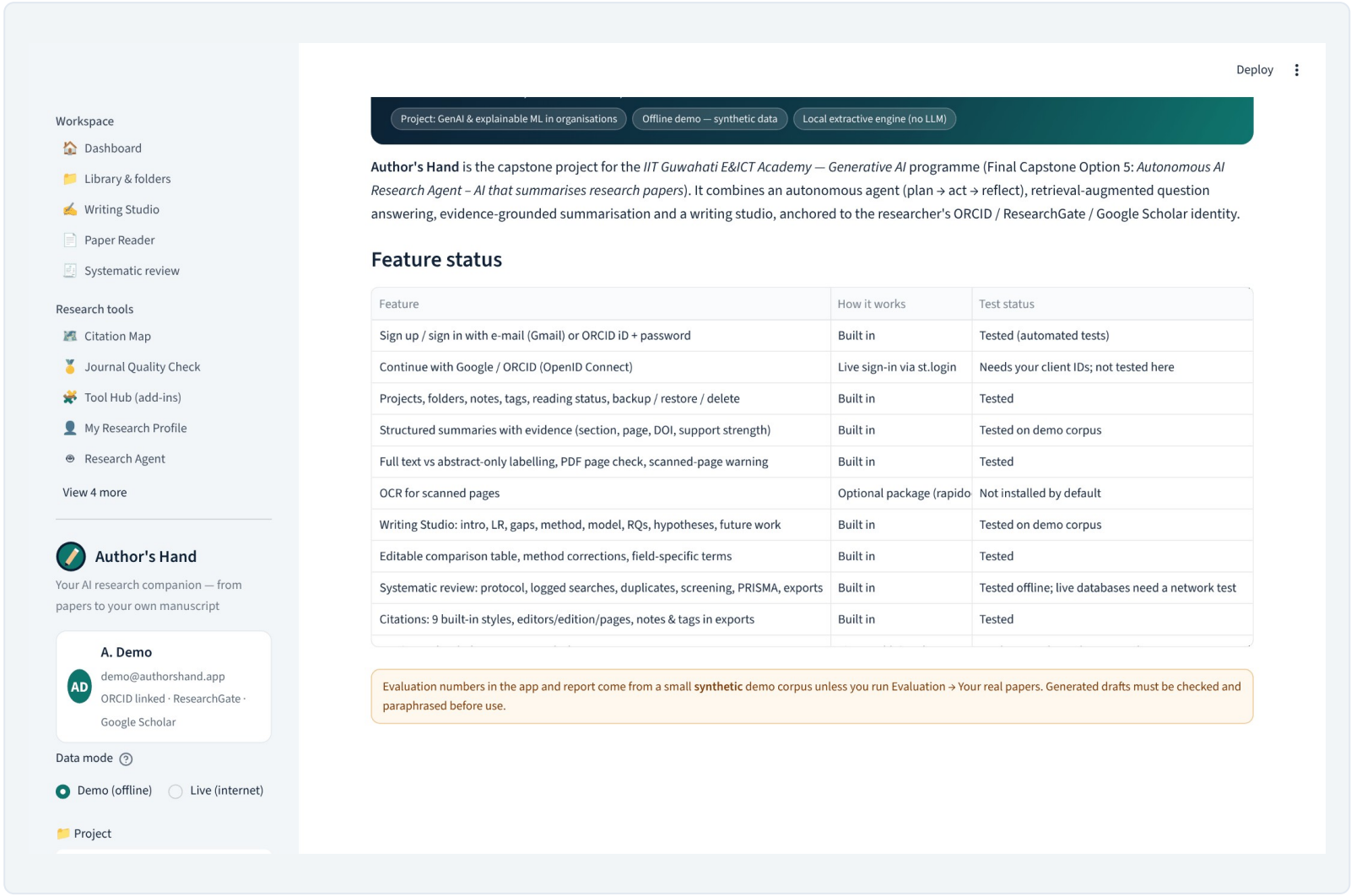
- Table of every data item, where it is stored and where it goes
- API keys kept in session memory only
- Identity discovery only on request
- Free local engine sends no paper text

Your data: backup, restore, delete



- Download the whole account as JSON
- Restore into any installation
- Delete account and all data (type DELETE)
- Unlink ORCID, ResearchGate, Scholar, Zotero

About: honest feature status



- Each feature marked built-in, live API, link or file exchange
- Test status for each
- Synthetic-data warning repeated

Link to the Generative AI course, and my contribution

Prompting & structured output 9-field summaries, Writing Studio drafts, JSON parsing

LangChain one chat-model interface; tools exported as StructuredTools

Embeddings & RAG TF-IDF ⊕ dense retrieval, cited answers, evidence grounding

Agents planner → tool calls → faithfulness gate → retry → trace

Evaluating generative output ROUGE vs LEAD-3, faithfulness, probes, human ratings

Deployment Streamlit, FastAPI, Docker, Windows app, Play Store kit

MY CONTRIBUTION (student to complete)

- Problem choice and requirements: [write here]
- Design decisions I made: [write here]
- Testing on my own papers and what I found: [write here]
- Interpretation of results and limitations: [write here]

AI-assistance disclosure: parts of the code and text were drafted with an AI assistant, then reviewed, tested and edited by the student.

Challenges we hit, and how we solved them

ResearchGate has no public API

Scraping breaches its terms. We link the profile URL (auto-found on ORCID) and import the user's own BibTeX/RIS export, then de-duplicate.

Hallucinated numbers in summaries

Claim-level grounding with a faithfulness score, a self-verification pass and a strict-mode retry loop.

Messy PDFs and inconsistent headings

Canonical section mapping (11 sections, 60+ heading variants), wrapped-title detection, de-hyphenation, unstructured fallback.

Same work from many sources

Merge by DOI, then fuzzy title (≥ 0.92); keep the richest fields and record provenance.

Cost and access to LLMs

Local extractive engine by default; one LangChain layer for Claude, OpenAI, Groq, Gemini and Ollama.

Most AI tools have no public API

Litmaps, SciSpace, JANE etc. are attached honestly: pre-filled launch links and BibTeX/RIS exchange; Zotero, ORCID, OpenAlex, DOAJ via real APIs.

Next: from capstone to research-office tool

Read tables & figures

Multimodal parsing so numbers in tables are summarised and grounded too.

Stronger verification

LLM-judge / NLI entailment in place of TF-IDF support; SciFact & arXiv benchmarks.

Citation snowballing

Expand reviews through references and citing papers automatically.

Shared team projects

Co-author workspaces with roles and comments on top of the new accounts.

Real-paper study

Run the built-in protocol on 30+ genuine PDFs with human ratings and report failures.

Multilingual papers

Summarise non-English literature with cross-lingual retrieval.

Deliverables: project.ipynb · app.py (Streamlit) · api.py (FastAPI) · report.pdf/.pptx · requirements.txt · README.md · Windows app · mobile/ Play Store kit · Dockerfile · 33 unit tests